Technische Hochschule **Rosenheim**

Fakultät für Informatik

Wirtschaftsinformatik

Konzeption und Entwicklung eines KI-gestützten Chatbots zur automatischen Beantwortung von Supportanfragen unter Verwendung interner Datenquellen

Bachelor Thesis

von

Atakan Sentürk

Datum der Abgabe: 05.08.2024
Erstprüfer: Prof. Dr. Marcel Tilly
Zweitprüfer: Prof. Dr. Gerd Beneken

Eigenständigkeitserklärung / Declaration of Originality

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken (dazu zählen auch Internetquellen) entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht.

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Rosenheim, den 05.08.2024

Atakan Sentürk

Kurzfassung

Diese Bachelorarbeit untersucht die Entwicklung und Leistungsfähigkeit moderner Chatbots unter Verwendung verschiedener Large Language Models. Der Schwerpunkt liegt auf der Konzeption und der prototypischen Implementierung eines Chatbots, sowie dem anschließenden Vergleich von LLMs innerhalb des entwickelten Systems.

Die Konzeption zeigte die Vielzahl aktueller Möglichkeiten zur Chatbot-Entwicklung. Die Implementierung wurde mit dem Framework Langchain durchgeführt, wodurch die Benutzerfreundlichkeit moderner Entwicklungstools deutlich wurde. Der resultierende Chatbot wurde erfolgreich in Google Chat integriert.

Im Anschluss wurden vier führende LLMs (Llama3, Phi-3, Mistral und Gemma 2) verglichen. Der Vergleich erfolgte durch eine quantitative Analyse mittels des Frameworks RAGAS, sowie einer qualitative Bewertung durch Fragebögen. In beiden Vergleichsansätzen erwies sich Llama3 als das beste LLM.

Diese Arbeit liefert wertvolle Einblicke in die praktische Umsetzung und Leistungsfähigkeit moderner Chatbot-Technologien und bietet eine Grundlage für zukünftige Entwicklungen in diesem Bereich.

Schlagworte: Chatbot, Generative KI, Large-Language-Modell, Prototyp, Retrieval-Augmented-Generation, Vektordatenbanken

Inhaltsverzeichnis

1 Einführung **1**
- 1.1 Motivation . . . 1
- 1.2 Zielsetzung . . . 2

2 Grundlagen **3**
- 2.1 Chatbots . . . 3
  - 2.1.1 Turing Test . . . 3
  - 2.1.2 ELIZA . . . 3
  - 2.1.3 Parry . . . 4
  - 2.1.4 ALICE . . . 4
  - 2.1.5 Siri, Google Assistant, Alexa . . . 5
  - 2.1.6 ChatGPT, Claude, Gemini . . . 5
- 2.2 Künstliche Intelligenz . . . 5
  - 2.2.1 Schwache künstliche Intelligenz . . . 7
  - 2.2.2 Starke künstliche Intelligenz . . . 7
  - 2.2.3 Natural Language Processing . . . 7
  - 2.2.4 Neuronale Netzwerke . . . 8
  - 2.2.5 RNN . . . 10
  - 2.2.6 LSTM . . . 10
  - 2.2.7 Encoder-Decoder Architektur . . . 11
  - 2.2.8 Attention Mechanismus . . . 11
- 2.3 LLMs . . . 12
  - 2.3.1 Transformer . . . 12
  - 2.3.2 Tokenizer . . . 14
  - 2.3.3 Training . . . 15
  - 2.3.4 Inferenz & Generation . . . 16
  - 2.3.5 Limitationen & bekannte Probleme . . . 17
  - 2.3.6 Prompting . . . 18
- 2.4 Vektor Datenbanken . . . 19
- 2.5 Embedding Modelle . . . 19
- 2.6 RAG . . . 20
  - 2.6.1 Naive RAG . . . 20
  - 2.6.2 Advanced RAG . . . 21
- 2.7 llama.cpp . . . 21
- 2.8 Ollama . . . 22
- 2.9 KI Frameworks . . . 23
  - 2.9.1 Haystack . . . 23
  - 2.9.2 Langchain . . . 23
  - 2.9.3 LlamaIndex . . . 24
- 2.10 Huggingface & MTEB . . . 24
- 2.11 Google Chat . . . 25

3 Anforderungsanalyse **26**
3.1 Ausgangslage . . . 26
3.2 Anforderungen . . . 27
3.2.1 A1 Selbst Hosting . . . 27
3.2.2 A2 Integration in Google Chat . . . 27
3.2.3 A3 Gruppenchat-Funktionalität . . . 27
3.2.4 A4 Nutzung von Unternehmensdaten . . . 27
3.2.5 A5 Erweiterbarkeit der Wissensdatenbank . . . 28
3.2.6 A6 Deutschsprachige Interaktion . . . 28
3.2.7 A7 Logische & schlüssige Argumente . . . 28
3.2.8 A8 Speicherung von Anfragedaten . . . 28

4 Konzeption **29**
4.1 Auswahl der Modelle . . . 29
4.1.1 Auswahl der Chat LLMs . . . 29
4.1.2 Auswahl der Embedding Modelle . . . 30
4.2 Auswahl der Vektor Datenbank . . . 30
4.3 Architektur des Systems . . . 31
4.3.1 Verbindungen . . . 31
4.3.2 Importer . . . 31
4.3.3 Chatbot . . . 32
4.3.4 Server . . . 33
4.3.5 Anbindung Google Chat . . . 34
4.3.6 Erfüllung der Anforderungen . . . 35

5 Entwicklung **36**
5.1 Vorbereitung . . . 36
5.1.1 Modelle . . . 36
5.1.2 Google Cloud . . . 36
5.1.3 Datenbank . . . 36
5.2 System . . . 36
5.2.1 Importer . . . 37
5.2.2 Chatbot . . . 39
5.2.3 Google Cloud API . . . 41
5.2.4 Post Endpunkt . . . 42
5.2.5 Server Konfiguration . . . 44
5.2.6 Google Chat API Konfiguration . . . 44
5.2.7 Chatbotinteraktion . . . 44

6 Vergleich **45**
6.1 Evaluierung . . . 45
6.2 RAGAS Vergleich . . . 45
6.2.1 Testset . . . 45
6.2.2 Evaluation . . . 46
6.2.3 Ergebnisse . . . 48
6.3 Umfrage Vergleich . . . 48
6.3.1 Fragebogen . . . 49
6.3.2 Fragebogen Ergebnisse . . . 49

7 Fazit **51**
7.1 Zusammenfassung Konzeption und Entwicklung 51
7.2 Zusammenfassung Bewertung 51
7.3 Ausblick 51

A Anhang **53**

Literaturverzeichnis **58**

Abbildungsverzeichnis

1.1 ChatGPT und Claude Interesse (Google Trends Data vom 26.06.2022 bis 26.05.2024) 2

2.1 Neuron eines Neuronalen Netzwerks 9
2.2 Neuronales Netz 10

4.1 konzeptionelle Architektur 31

5.1 Konkrete Architektur 37
5.2 Aufbau der Chain 39
5.3 RunnableWithMessageHistory 43

6.1 Ragas Ergebnisse 48

A.0 JSON Nachricht der Google Chat API 55
A.1 Asynchrones Message Objekt 55
A.2 Darstellung Threadinteraktion 56
A.3 Durchschnittliche Bewertungen von Llama3 und Phi3 57
A.4 Github Star Historie der Frameworks Langchain, Haystack und LlamaIndex . . . 57

1 Einführung

Im Jahr 2022 konnten Unternehmen durch den Einsatz von Chatbots 11 Milliarden Dollar einsparen. Mit der fortlaufenden Entwicklung und zunehmenden Verbreitung dieser Technologie ist zu erwarten, dass diese Einsparungen in Zukunft weiter ansteigen werden [Fok].

Durch die steigende Popularität von Chatbots wie ChatGPT von OpenAI [Cha] und Claude von Anthropic [Cha] in den letzten Jahren, wie in 1.1 erkennbar, werden die Fortschritte, die im Bereich der künstlichen Intelligenz erreicht werden, immer bekannter und der Nutzen den KI Applikationen im Leben von Menschen haben wird dadurch immer erweitert.

Chatbots sind inzwischen in der Lage, diverse Aufgaben im Bereich der Kundekommunikation zu übernehmen, weshalb Unternehmen den ersten Schritt ihres Kundensupports durch diese ersetzen. Dies wird bei manchen Unternehmen über ein Chat Interface auf ihrer Website eingebaut. Über dieses Interface können die Nutzer fragen stellen und Probleme lösen lassen, falls keine Lösung gefunden werden kann, werden sie dann an den Kundensupport weitergeleitet [Dis].

1.1 Motivation

Die Motivation für diese Bachelorarbeit entspringt der Erkenntnis, dass trotz konstanter Forstschritte im Bereich der Large Language Models und Chatbots, weiterer Forschungs- und Entwicklungsbedarf besteht.

Die Inhalte dieser Bachelorarbeit wurden zusammen mit der Techdivision GmbH erarbeitet. Die Techdivision hat firmenintern Support Chats, in denen die Mitarbeiter der Firma, bei technischen Problemen Fragen stellen können und diese vom IT-Support Team beantwortet werden.

Einige dieser Fragen sind wiederkehrende Fragen, die mithilfe der firmeninternen Dokumentation gelöst werden könnten. Ein typisches Beispiel ist die Frage, warum eine Anmeldung bei Bitwarden mit dem Master-Passwort nicht möglich ist. Um die Ressourcen des IT-Teams effektiver zu nutzen und die Probleme der Fachbereiche schneller zu lösen, soll ein Chatbot erstellt werden. Dieser Chatbot soll Zugriff auf relevante Unternehmensdaten erhalten und dadurch in der Lage sein, Fragen innerhalb des Support-Chats automatisch und präzise zu beantworten.

Diese Arbeit zielt darauf ab, einen Beitrag zur Überbrückung der Lücke zwischen theoretischen Konzepten und praktischer Anwendung zu leisten. Durch die konzeptionelle Entwicklung eines Chatbots und die anschließende prototypische Implementierung wird ein Ansatz verfolgt, der es ermöglicht, verschiedene LLMs im Kontext realer Anwendungsszenarien zu evaluieren.

Der Vergleich von vier unterschiedlichen LLMs innerhalb des entwickelten Prototyps bietet die Gelegenheit, die Anwendbarkeit dieser Modelle zu untersuchen. Diese vergleichende Analyse soll zur Ermittlung des am besten geeigneten Modells für diesen Anwendungsfall beitragen.

Darüber hinaus adressiert diese Arbeit die wachsende Nachfrage nach praxisorientierten Lösungen im Bereich der KI-gestützten Kommunikation. In einer Zeit, in der Unternehmen und Organisationen zunehmend auf KI-Systeme setzen, um Effizienz zu steigern und Kundenerlebnisse zu verbessern, liefert diese Arbeit konkrete Erkenntnisse für die Entwicklung und den Einsatz moderner Chatbots.

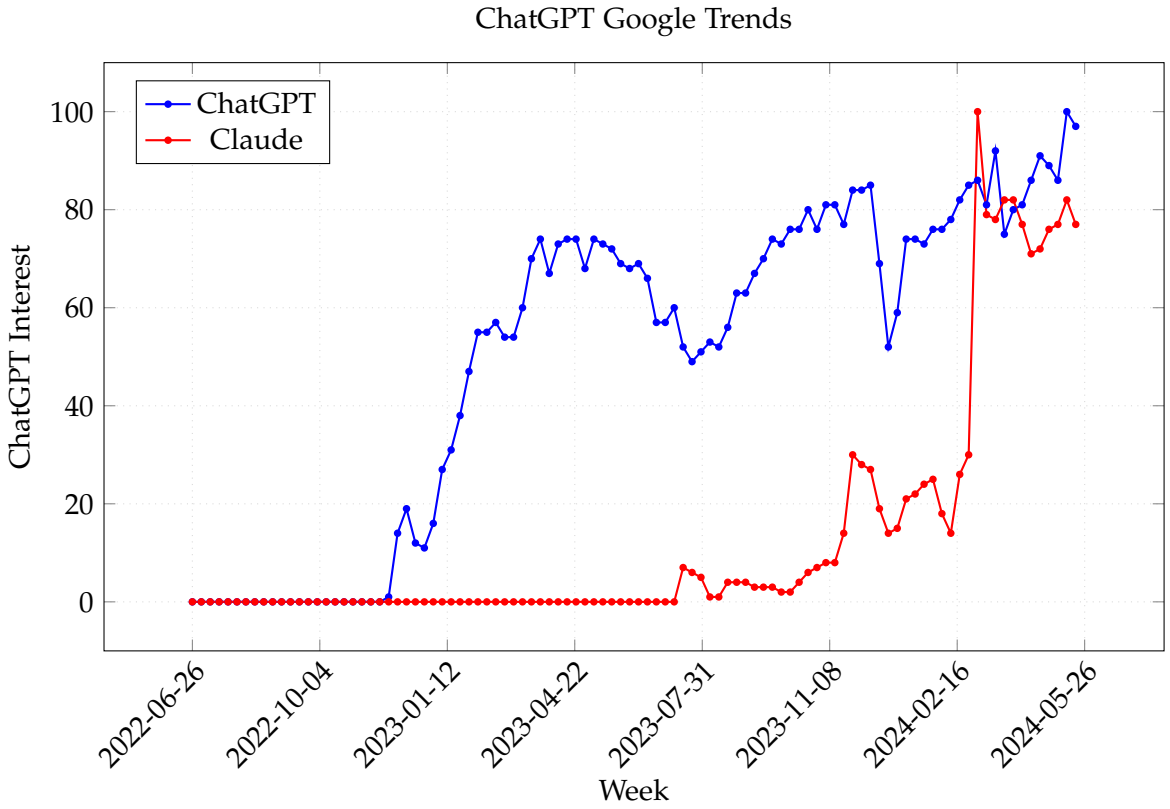


Abbildung 1.1 ChatGPT und Claude Interesse (Google Trends Data vom 26.06.2022 bis 26.05.2024)

1.2 Zielsetzung

Das Ziel der Arbeit besteht darin, einen Chatbot zu konzipieren und einen Prototyp zu entwickeln, der mit Hilfe der Daten der internen Datenquellen der Techdivision Support leistet und Fragen beantwortet. Der Chatbot soll innerhalb von Google Chat integriert werden. Dabei ist es wichtig, dass der Chatbot, statt in Einzelchats, in einer Chat Gruppe läuft. Diese beinhaltet neben den Support Suchenden, ebenfalls Mitarbeiter des jeweiligen Services. Auf diese Weise kann, falls notwendig, extern eingegriffen werden.

Hierfür wird im Rahmen dieser Arbeit ein Konzept für einen Chatbot erstellt, welches prototypisch implementiert wird. Dazu werden über die Implementierung vier verschiedene LLMs verwendet und miteinander verglichen, um das beste für diesen Anwendungsfall zu ermitteln. Der Vergleich wird in zwei Stufen durchgeführt. In der ersten Stufe werden die LLMs über das Framework Ragas verglichen. Im zweiten Schritt werden die zwei besten LLMs aus dem Ragas Vergleich, über einen Fragebogen Vergleich von Mitarbeitern der TD bewertet und miteinander verglichen.

2 Grundlagen

2.1 Chatbots

Ein Chatbot ist ein Dialogsystem, das den Dialog zwischen einem Menschen und einem technischen System erlaubt [Ada20]. Von diesen gibt es verschiedene Arten. Diese unterscheiden sich in der Art und Weise, wie sie auf die Eingaben des Nutzers antworten. Chatbots, wie OpenAIs ChatGPT und Anthropics Claude, haben einen Bereich zur Texteingabe und einen Bereich, in dem sie ihre Ausgabe darstellen können. Mit diesen kann heutzutage über die natürliche menschliche Sprache kommuniziert werden und sie versuchen auf dieselbe Art und Weise zu antworten.

Die Generation des Textes kann in drei verschiedene Kategorien untergeordnet werden: abfragebasiert, generationsbasiert und regelbasiert [Ada20]. Abfragebasierte Chatbots haben eine Wissensdatenbank in der sie nach dem am besten zur Eingabe passenden Eintrag suchen. Beim generationsbasierten Ansatz generieren sie ihre Antworten selbst. Regelbasierte Chatbos antworten nach vorher in ihrem Skript bestimmten Regeln. Hierbei vergleichen sie die Eingabe mit ihren vorher definierten Regeln und transformieren die Eingabe, falls möglich, in eine vorher definierte Satzstruktur. [Ada20] Im nächsten Schritt wird darauf eingegangen, wie sich Chatbots mit der Zeit verändert haben.

2.1.1 Turing Test

Im Jahr 1950 hat Alan Turing den Turing Test vorgeschlagen, bei dem es darum geht zu erkennen, ob eine Maschine das gleiche Denkvermögen hat wie ein Mensch. Bei diesem Test werden einem Tester zwei Gesprächspartner vorgelegt, von denen einer ein Mensch und der andere eine Maschine ist. Der Tester führt dann ein Gespräch mit beiden Gesprächspartnern und befragt diese. Nach der Befragung wird überprüft, ob der Tester in der Lage ist, zu sagen, welcher der Partner eine Maschine ist. Falls der Tester nicht in der Lage ist zu erkennen, welcher der Partner die Maschine ist, gilt der Turing Test als bestanden und die Maschine ist laut Turing intelligent [LaC].

2.1.2 ELIZA

Die Geschichte der Chatbots beginnt in den 1960er Jahren, mit dem ersten aus dem Jahr 1966. Damals waren die Chatbots regel basiert. Der Name des Chatbots war ELIZA und er wurde von Joseph Weizenbaum am MIT entwickelt. Weizenbaum hat fünf Grundprobleme aufgestellt, welche er für die Entwicklung beachten musste [Wei66].

Die Probleme waren:

1. Die Identifikation von Stichwörtern
2. Das Finden des minimalen Kontexts
3. Die Auswahl der geeigneten Transformation

4. Generation von Antworten bei Abwesenheit von Stichworten

5. Bearbeitung der Eliza Skripte

Die Regeln, die ELIZA beachten muss, wurden ihr über Skripte gegeben. Sie hat versucht die menschliche Sprache zu imitieren, indem sie den eingegebenen Text nach Stichwörtern durchsucht. Stichwörter sind Wörter, denen besonders hohe Wichtigkeit laut dem ELIZA Skript gegeben wurde. Jedes Stichwort hatte einen Rang, welcher die Wichtigkeit des jeweiligen Stichworts im Vergleich zu anderen Stichworten festgelegt hat. Nachdem die Stichwörter entdeckt wurden, betrachtet ELIZA die Transformations Regeln, die in ihrem Skript bestimmt wurden [Wei66].

Die Transformations Regeln bestehen aus einer Zerlegungs und einer Zusammensetzungs-Phase. Es gibt verschiedene Regeln für beide Phasen. Anhand der Syntax des Satzes wird identifiziert, welche Regel für die Zerlegung in Frage kommt und anhand dieser wird dann die Zusammensetzung ausgemacht. Dieser Zusammengesetzte Satz wird im Anschluss ausgegeben Im Fall, dass kein Stichwort gefunden wurde, wird entweder eine inhaltslose Antwort oder eine frühere Antwort zurück gegeben [Wei66].

ELIZA verhält sich wie ein Psychotherapeut. Das heißt, der Nutzer beginnt das Gespräch und der Chatbot greift Punkte aus den Eingaben des Nutzers auf, ohne großen Zusammenhang mit Themen außerhalb dieses Gesprächs und ihrer Regeln. Daraus lässt sich schließen, dass ELIZA keinen Kontext kennt, sondern, je nach seinem aktuellen Skript auf den aktuellen Satz antwortet [Wei66].

2.1.3 Parry

Der nächste Chatbot, Parry, wurde an der Stanford University entwickelt. Während Eliza einen Psychotherapeuten imitiert hat, hat Parry einen Patienten mit paranoider Schizophrenie imitiert. Dieser Bot war im Jahr 1979 Teil eines Experiments, der einem Turing Test (siehe 2.1.1) gleicht. Hier wurden verschieden Psychotherapeuten vor Bildschirme gesetzt und mussten erraten, ob die gegenüber gestellten Aussagen von einem echten schizophrenen Patienten kommen oder von einem Chatbot. Teil des Experiments waren 5 Psychiater, die jeweils 2 Patienten zur Beurteilung bekommen haben.

Von den 5 Teilnehmern lag nur einer bei beiden richtig. Einer lag bei beiden falsch, ein weiterer dachte beide Patienten sind echte Patienten und die letzten zwei dachten, dass beide Patienten Chatbots sind [Ada20].

Das Problem hierbei war jedoch, dass der Pool an Probanden mit 5 Psychiatern sehr klein war. Dazu kommt noch der Fakt, dass Patienten mit Schizophrenie ein gewisses Level an Unverständlichkeit in ihren Konversationen haben, welches die Beurteilung, ob es generell ein guter Chatbot ist erschwert. Heute wird Parry eher als ein schwacher Chatbot gesehen, da er nicht von Konversationen lernen kann, keinen richtigen Kontext hat und sehr langsam ist. Natürlich ist ein Teil davon abhängig von den technischen Limitationen die es damals gab, aber es ist dennoch wichtig zu beachten [Ada20].

2.1.4 ALICE

1995 wurde der Internet Chatbot ALICE veröffentlicht. Dies war der erste Chatbot der im Internet zu finden war und er wurde als Open Source Software veröffentlicht. Im Laufe der Jahre wurde das Projekt mehrmals mit verschiedenen Preisen ausgezeichnet, einer dieser war beim Loebner Preis. Der Loebner Preis ist ein Wettbewerb, bei dem eine Jury entscheidet,

welches der teilnehmenden Programme am menschenähnlichsten ist [Hua]. Dieser Chatbot wurde von Weizenbaums ELIZA inspiriert und war selbst die Inspiration für den Film "Her", in der die Beziehung zwischen Menschen und Maschinen betrachtet wird [Mor13]. Alice hatte eine viel größere Sammlung an Stichwörtern, auf die sie reagieren konnte als ELIZA, aber sie hat auf die selbe Art und Weise Stichwörter in den Eingaben des Nutzers gesucht und dann je nach ihren Datenbankeinträgen darauf geantwortet [Ada20, Wal09].

2.1.5 Siri, Google Assistant, Alexa

Nach diesen anfänglichen Versuchen, welche vorallem im Bereich der Psychologie entwickelt wurden und die regelbasiert waren, wurden immer weiter Fortschritte gemacht.

Eine große Neuerung im Jahr 2010 war Siri, ein persönlicher Assistent auf dem eigenen Handy. Siri, welches nur auf Apple Geräten zur Vefügung steht, hilft den Nutzern, welche über Text oder Sprachbefehle mit dem Assistenten kommunizieren können, mit verschiedenen Aufgaben. Unter anderem können die Nutzer mittels Siri Erinnerungen setzen, Nachrichten senden und Anrufe tätigen [Sir].

Sprachassistenten wie Siri gibt es auch auf anderen Plattformen, wie auf Samsung Galaxy Geräten mit Galaxy AI und verschiedenen Android Geräten mit Google Assistant. Diese Assistenten haben ähnliche Funktionen wie Siri und versuchen dem Nutzer zu helfen[Goo, Gal].

SmartHome Geräte, wie Smarte Lampen und Steckdosen lassen sich mit Hilfe von diesen Sprachassisten kontrollieren.

Manche Sprachassistenten lassen sich über extra Geräte, die für diesen Nutzen hergestellt wurden, bedienen. Ein Beispiel hierfür ist Amazon Alexa, auf dem Echo Dot. Der Echo Dot ist ein kleiner Lautsprecher, welcher mit dem Internet verbunden ist. Durch diesen kann mit Alexa kommuniziert werden [Ama].

Diese Sprachassisten werden immer weiter entwickelt und bekommen dadurch neue Funktionen, mehr Sprachen oder die Fähigkeit die menschliche Sprache besser zu interpretieren [Sir].

2.1.6 ChatGPT, Claude, Gemini

Wie in 1 bereits gezeigt, steigen aktuell die Anfragen für Chatbots wie ChatGPT und Claude. Diese Chatbots basieren auf Large Language Models, welche in 2.3 genauer erklärt werden. Dies macht sie zu generationsbasierten Chatbots. Nutzer können teils über Apps [Cha24] und teils über die Website [Cla], des jeweiligen Services, mit diesen Bots kommunizieren.

Den Chatbots werden fragen gestellt, welche vom jeweiligen System beantwortet werden. Sie können nicht nur in Textform antworten, sondern auch durch Bild oder Audiogenerierung, wie bei ChatGPTs DALL·E Integration. Diese Chatbots sind nicht nur in der Lage Fragen zu beantworten, sondern sie können auch Code, einen Essay oder ein Film Script generieren.

2.2 Künstliche Intelligenz

Künstliche Intelligenz ist ein Teil der Informatik der sich damit beschäftigt, Aufgaben, die früher menschliche Intelligenz benötigt haben, von Maschinen bzw. Computern übernehmen zu lassen. Zu den Aufgaben gehören unter anderem:

- Erkennen von Mustern

- Treffen von Entscheidungen
- Verstehen von Sprache

Durch Fortschritte in der künstlichen Intelligenz konnten Forstschritte in all diesen Aufgabenbereichen erreicht werden. Die verschiedenen Aufgaben werden in verschiedenen Teilbereichen der künstlichen Intelligenz behandelt. Ein paar der Teilbereiche sind:

- Computer Vision
- Natural Language Processing
- Robotik

Nun ist die Frage wie Intelligenz definiert werden kann. Das ist keine einfache Aufgabe und nicht der Schwerpunkt dieser Arbeit. Im Kontext dieser Arbeit wird das Buch "Knowledge Science – Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten" von Lanquillong und Schacht und die Definition von Intelligenz daraus betrachtet. Laut Lanquillon und Schacht ist eine Möglichkeit ein paar Eigenschaften anzuschauen, welche an menschliches Verhalten angelehnt sind und davon ausgehend künstliche Intelligenz definieren. Diese Liste ist offen und nicht vollständig. Die Eigenschaften die im Buch genannt werden sind:

- Wahrnehmung
- Schlussfolgerung
- Handeln
- Planung und zielgerichtete Problemlösung
- Kommunikation
- Anpassungsfähigkeit und Lernen
- Autonomie
- Kreativität
- Reflexion und Bewusstheit
- Ästhetik
- Organisation

Die meisten Menschen weisen alle diese Eigenschaften auf, aber es gibt noch keine künstliche Intelligenz, die alle davon aufweisen kann. Da es noch kein System gibt, welches in diese Kategorie fällt, aber es bereits Systeme gibt, die ein paar dieser Eigenschaften aufweisen und dazu noch Probleme in eng definierten Problembereichen lösen können. Ist eine weitere Unterscheidung innerhalb des Bereichs der künstlichen Intelligenz nötig. Daraus formen sich die zwei Begriffe schwache künstliche Intelligenz und starke künstliche Intelligenz [Lan23, S. 5-7].

2.2.1 Schwache künstliche Intelligenz

Die schwache künstliche Intelligenz ist ein System, welches spezifische Probleme innerhalb einer bestimmten Domäne lösen kann. Diese Art der künstlichen Intelligenz wird darauf trainiert, Probleme innerhalb eines eng begrenzten Bereichs zu lösen, indem sie speziell auf diesen Bereich trainiert wird und im Laufe dessen dann die Regeln dieses Bereichs kennen lernt. Nachdem es trainiert wurde, kann die schwache künstliche Intelligenz durch die beim Training erlernten Regeln und Muster, Probleme innerhalb der Domäne in der es trainiert wurde lösen. Es kann dieses Wissen aber nicht auf andere Problembereiche übertragen und dadurch Probleme in dieser neuen Domäne lösen.

Ein Beispiel für eine schwache künstliche Intelligenz wäre ein System, welches darauf trainiert wurde Texte von einer Sprache in einer andere zu übersetzen. Das System kann vielleicht innerhalb dieser Domäne die Aufgabe sehr gut erfüllen, was heißt, dass sie die Texte mit hoher Genauigkeit in die Zielsprache übersetzen kann, ohne den Verlust von Informationen, aber egal wie gut es in dieser Domäne ist, bleibt das System auf diese Domäne beschränkt. Das heißt, es kann das erlente Wissen, nicht auf unverwandte Aufgaben übertragen. [Lan23].

2.2.2 Starke künstliche Intelligenz

Starke künstliche Intelligenz oder Allgemeine künstliche Intelligenz, wäre ein System, welches nicht nur die in 2.2 genannte Liste an Eigenschaften aufweisen könnte, sondern auch in der Lage wäre, Probleme zu lösen, ohne explizit für diese trainiert worden zu sein. Die starke künstliche Intelligenz, wäre in der Lage ihr erlerntes Wissen zu verallgemeinern und es dadurch, ähnlich wie ein Mensch, auf neue Anwendungsbereiche anzuwenden. Darüber hinaus, wäre eine starke künstliche Intelligenz ebenfalls in der Lage sich selbst weiterzuentwickeln und zu lernen, ohne das menschliches Eingreifen erforderlich wäre. Diese Fähigkeit würde bedeuten, dass starke künstliche Intelligenz, durch diese Selbstverberbesserungsfähikeit in der Lage wäre ihr eigenes System und ihre eigenene Algorithmen zu optimieren und somit irgendwann die menschliche Intelligenz zu übertreffen [Lan23].

Solch ein System würde nicht nur durch seine Problemlösungsfähigkeit ausgezeichnet sein, sonder ebenfalls durch kreatives Denken und emotionale Intelligenz. Dadurch wäre sie in fast allen Bereichen einsetzbar, in denen diese Eigenschaften gefragt sind [Lan23].

Mit dem Aufstieg von künstlichen Intelligenzen im Auge der Öffentlichkeit, zeigen sich einige Menschen besorgt, was passieren würde, wenn solch ein System erschaffen werden würde. Es ist jedoch zu betonen, dass ein solches System noch nicht existiert und es ist davon auszugehen, dass dies auch in naher Zukunft nicht der Fall sein wird [Rai24].

2.2.3 Natural Language Processing

Natural Language Processing oder natürliche Spracherkennung, ist ein Teil des Machine Learnings der sich auf die Arbeit mit menschlicher, natürlicher Sprache fokussiert. Das Ziel von NLP ist es, Computern die Möglichkeit zu geben, menschliche Sprache zu verstehen und zu verarbeiten, um mit Menschen kommunizieren zu können. Dies schließt Kommunikation in Form von Audio als auch über Text mit ein [Wha21].

NLP ermöglicht es Computern, durch Kombination verschiedener Machine Learning Verfahren und anderer linguistischer Verfahren menschliche Sprache zu verstehen und diese auch wiederzugeben. Diese Systeme können, Texte zusammenzufassen, sie analysieren, ihre

Bedeutung zu verstehen und mit Hilfe dieser Fähigkeiten dann sinnvolle und passende Antworten zu generieren. Anwendungen die NLP nutzen sind vielfältig, Systeme wie Chatbots und Übersetzungssoftware implementieren ebenfalls NLP Verfahren [Wha21].

Da Sprache oft kontextabhängig ist und manche Sprachen, wie Japanisch komplette Satzteile weg lassen, wenn diese durch den Kontext erkenntlich sein sollten, ist es ein wichtiger Teil des Natural Language Processing, in der Lage zu sein kontextuelle Informationen zu berücksichtigen. Dadurch kann das System Kontext verstehen und richtig interpretieren. Dies wird über verschiedene Methoden erreicht, wie Tokenisierung, Parsing und Named Entity Recognition [Wha21].

NLP stellt eine wichtige Schlüsselstelle in der Human-Computer Interaction (zu deutsch Mensch-Computer-Interaktion) dar. Sie hilft nicht nur dabei menschliche Kommunikation mit Computern einfacher und schneller zu machen, sondern gibt auch Menschen, die aufgrund von körperlichen Problemen ein Handy nicht ohne Probleme bedienen können, die Möglichkeit mit diesen Geräten zu interagieren [Wha21].

2.2.4 Neuronale Netzwerke

Neuronale Netze sind ein Teil des Machine Learnings, welcher versucht sich stark an dem Aufbau eines biologischen Gehirns zu orientieren. Dementsprechend bestehen sie aus Tausenden oder sogar Millionen Neuronen, welche durch Knotenpunkte dargestellt werden. Diese Neuronen sind über verschiedene Reihen miteinander verbunden. Dieses Schema ist in 2.2 dargestellt [Goo16, S. 5 - 7].

In 2.1 ist ein Neuron dargestellt. Jeder Eingang an einem Neuron hat eine Gewichtung. Wenn die Gewichtung eines Eingangs 0 ist, dann heißt es, dass zwei Knoten keine Verbindung miteinander haben. Wenn die Gewichtung positiv ist, dann hat es eine bestärkende Wirkung und, falls die Gewichtung negativ ist, dann hat es eine hemmende Wirkung [Goo16, S. 5 - 7].

Jedes Neuron hat zudem einen sogenannten Bias. Durch den Bias hat das Neuron eine Grundaktivität, welche, ebenfalls während dem Training angepasst werden kann [Kem23]. Das aktuelle Neuron summiert dann alle gewichteten Werte und addiert den Bias. Wie in dieser Formel dargestellt:

$$\sum_i w_i x_i + b$$

Der summierte Wert wird durch die Aktivierungsfunktion transformiert und das Ergebnis wird dann dem nächsten Neuron in der nächsten Schicht weitergeleitet [Goo16, S. 5 - 7]. Typische Aktivierungsfunktionen [Kem23] sind:

- Sigmoid Function :
$$f(x) = \frac{1}{1 + e^{-x}}$$
- ReLu (Rectified Linear Unit):
$$f(x) = \max(0, x)$$
- Tanh:
$$f(x) = tanh(x)$$

Die Anzahl der Neuronen schwankt zwischen verschiedenen Neuronalen Netzen und auch innerhalb eines Neuronalen Netzes schwankt die Anzahl der Neuronen zwischen den verschiedenen Schichten. Bei der Eingabe Schicht hängt die Anzahl der Neuronane von

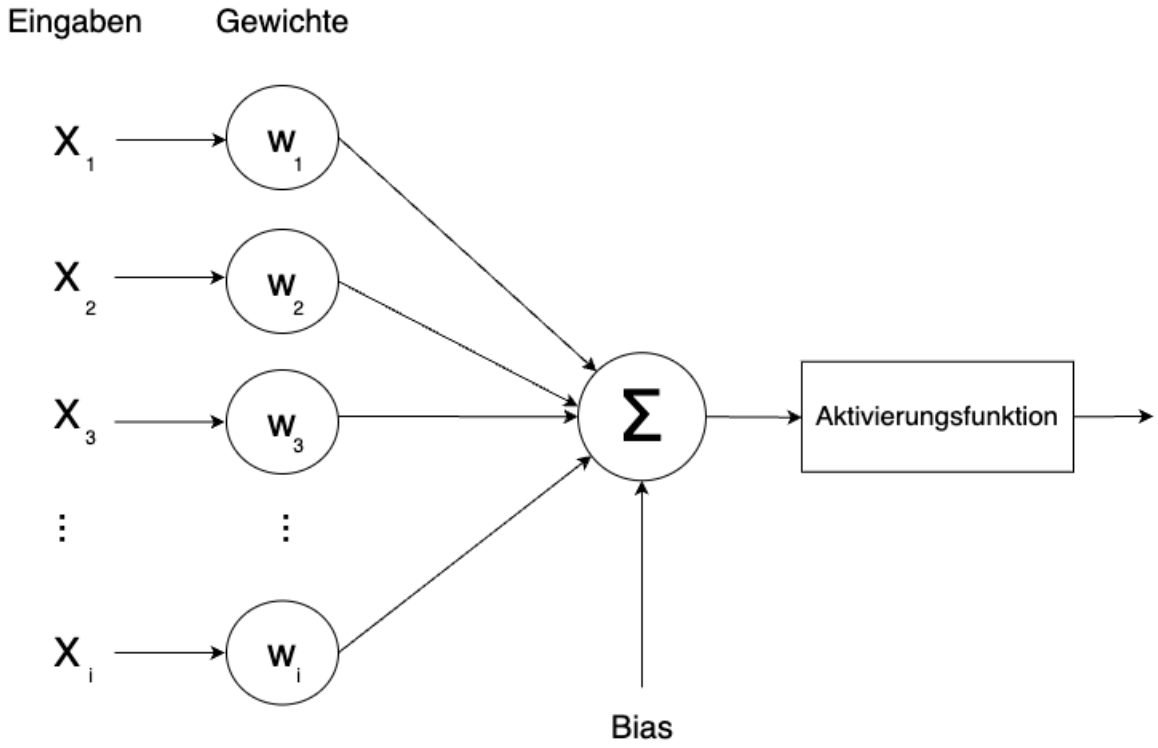


Abbildung 2.1 Neuron eines Neuronalen Netzwerks

den Trainingsdaten ab. Um genau zu sein, kommt es auf die Anzahl der Features an, die die Trainingsdaten beinhalten. Bei einem Neuronalen Netz, welches mit Daten aus einer normalen Tabelle trainiert wird, ist das beispielsweise die Anzahl der Spalten, bei einem Neuronalen Netz, welches mit Bildern trainiert werden soll, könnten das die einzelnen Pixel des jeweiligen Bildes sein [Kri22].

Es gibt X verschiedene verborgene Schicht in einem Neuronalen Netzwerk. Die Anzahl der Schichten und die Anzahl der Neuronen pro verborgener Schicht kann sich unterscheiden. Es gibt hierbei keine allgemeingültige Größe, die für alle Fälle passt. Stattdessen sind hier Erfahrung und Experimentierung wichtig [][S. 129]heatonIntroductionNeuralNetworks2008.

Die Anzahl der Neuronen in der Ausgabe Schicht ist davon abhängig, welche Art von Modell betrachtet wird. Die zwei Arten die hier besprochen werden sind Regressoren und Klassifikatoren [Hea08, S. 128].

Regressoren sagen für einen Eingabewert einen oder mehrere kontinuierliche Ausgabewerte hervor [Goo16, S. 100].

Beispiele hierfür sind:

- Gesichtserkennung
- Stimmerkennung
- Spamfilter in emails

Klassifikatoren sind dafür zuständig die Eingabe einer bestimmten Klasse zuzuordnen [Goo16, S. 101]. Die Anzahl der Neuronen hängt von der Anzahl der möglichen Klassen ab [O'S15].

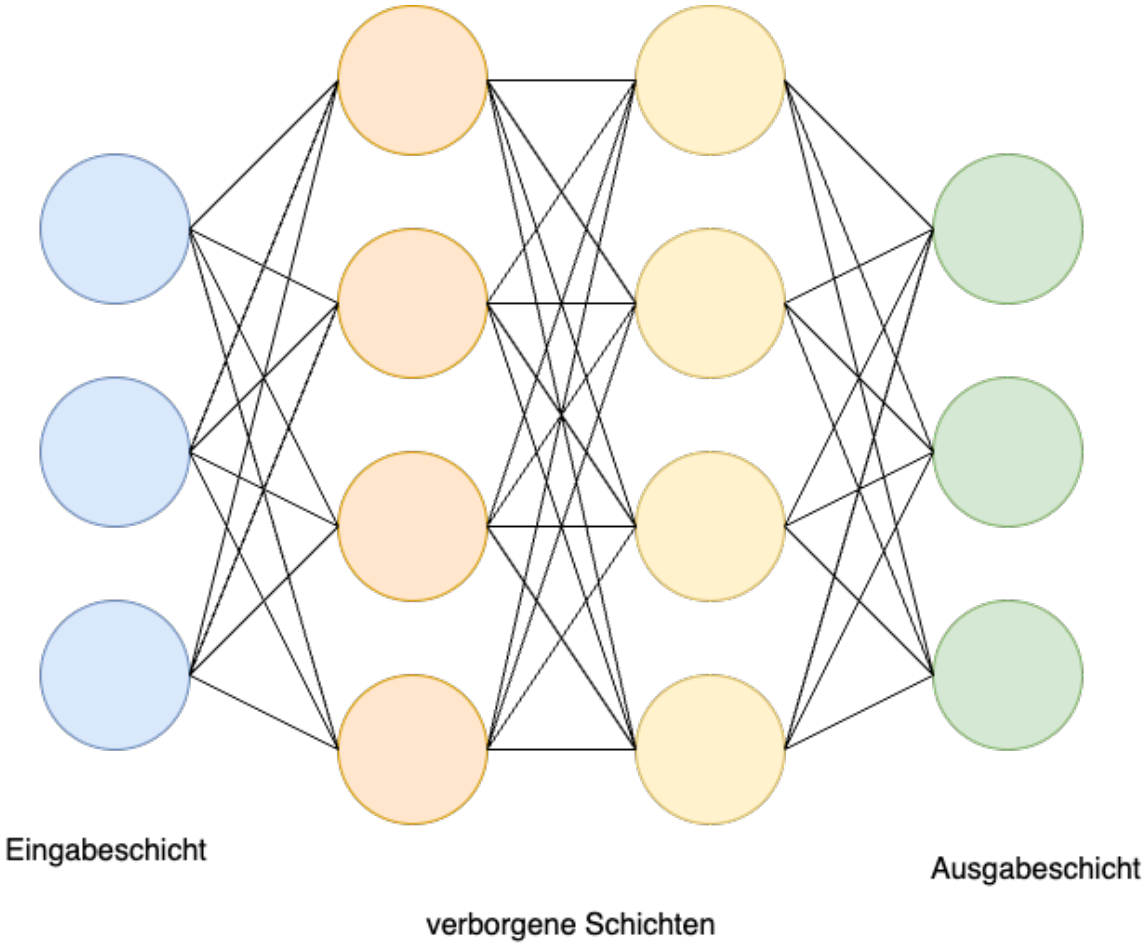


Abbildung 2.2 Neuronales Netz

2.2.5 RNN

Bevor auf die Transformer Architektur eingegangen werden kann, ist es wichtig, die vorangegangenen Architekturen zu verstehen. Dies ist wichtig, um zu verstehen, wie es zu den heutigen Architekturen gekommen ist und um zu erkennen, welche Vorteile diese bieten.

Bei der Weiterentwicklung von Neuronalen Netzen wurden Recurrent Neural Networks entwickelt. Bei diesen werden als Eingabe Sequenzen, die aufgeteilt werden, genutzt. Die einzelnen Teile werden dem Neuronalen Netz als Eingabe gegeben und während der Verarbeitung entsteht ein hidden State in dem RNN, welcher die Verarbeitung der nächsten Teile der Sequenzen beeinflusst [Whac].

Dadurch wird das Problem der Neuronalen Netze beseitigt, bei dem ein Neuronales Netz keine Erinnerungen und keinen Kontext hat und deshalb nur auf die aktuelle Eingabe reagieren kann.

2.2.6 LSTM

Ein Problem bei RNNs ist das Vanishing Gradient Problem, welches im Zusammenhang mit Backpropagation steht. Backpropagation ist ein Algorithmus, der zum Anpassen der Gewichte und des Biases von der Ausgabe zurückgehend bis zur Eingabe die Gradienten des

Fehlers berechnet und davon ausgehend durch Anpassungen der Parameter die Leistung des Modells verbessert. Das Vanishing Gradient Problem tritt auf, wenn Gradienten für zeitlich weit zurückliegende Eingaben entweder extrem klein oder extrem groß werden [Ven21].

Um dieses Problem zu lösen, wurde Long Short-Term Memory eingeführt. Dieser Ansatz ersetzt die rekurrenten Zellen des RNNs durch LSTM-Zellen. Diese Zellen haben nicht nur einen Hidden State, sondern auch einen Cell State, der für das Langzeitgedächtnis genutzt wird. Die LSTM-Zelle hat drei Gatter: ein Eingabe-Gatter, ein Ausgabe-Gatter und ein Gatter, welches für das Vergessen von Informationen zuständig ist [Ven21].

Dieser Ansatz mindert das Vanishing Gradient Problem und sorgt dafür, dass RNNs mit LSTM einen längeren Kontext effektiver zu verarbeiten als herkömmliche RNNs.

2.2.7 Encoder-Decoder Architektur

Diese Technologien haben sich damals vorallem im Kontext des Sequence-to-Sequence Learnings weiterentwickelt. Sequence-to-Sequence Learning oder kurz Seq2Seq Learning, bringt eine Sequenz variabler Länge in eine andere Sequenz variabler Länge [Sut14]. Beispiele hierfür wären Text Zusammenfassungen und Übersetzungen. Die Architektur aus der sich der Transformer (siehe 2.3.1) entwickelt hat, ist die Encoder - Decoder Architektur.

Die Encoder-Decoder Architektur besteht aus zwei RNNS. Wie ein Neuronales Netz funktioniert wurde bereits im Abschnitt 2.2.4 behandelt. Und RNNs wurden in 2.2.5 behandelt. Dadurch, dass die vorangegangenen Aktivierungen durch die Nutzung von RNNs ebenfalls beachtet werden, können Kontexte erkannt und Sätze können sinnvoll übersetzt werden [Sut14, Bah16].

Hierbei erhält der Encoder eine Eingabe Sequenz, welcher dann Kontext Vektoren erstellt. Diese werden vom Decoder in eine Ausgabe Sequenz umgewandelt. Das revolutionäre an dieser Technologie war damals, dass die Eingbae und Ausgabesequenzen variable Längen haben konnten. Der Kontext Vektor hatte jedoch eine feste Länge. Dieses Originale Encoder-Decoder Modell hatte trotz der Vorteile, die es geboten hat, jedoch einige Probleme [Sut14, Bah16]:

1. Sequenzielles Verarbeiten: RNNs und LSTMs verarbeiten Sequenzen sequentiell. Dadurch, dass die Daten nich parallel verarbeitet werden, kommt es zu längeren Trainingszeiten.

2. Abhängigkeiten: In RNNs ist es, aufgrund des Vanishing Gradient Problems, schwierig langfristige Abhängigkeiten zu erfassen.

3. Festes Kontextfenster: Da die meisten RNNs und LSTMs eine feste Kontextgröße haben, kann nur eine begrenzte Menge an Informationen als Kontext, zu einem bestimmte Zeipunkt, beachtet werden. Dadurch kann es zum Verlust von Informationen oder zur Verringerung der Genauigkeit der Antworten kommen. Besonders bei Sätzen die länger sind als die, in den Trainingsdaten, kann es hier zu Problemen kommen.

2.2.8 Attention Mechanismus

Die Idee auf die die Transformer aufbauen ist die des Attention Mechanismus, welcher 2014 in der Arbeit “Neural Machine Translation by Jointly Learning to Align and Translate” von Bahdanau et al. vorgestellt wurde. Zu Beginn wurde der Mechanismus genutzt um die in

2.2.7 beschriebene Encoder-Decoder Architektur zu verbessern, indem es vorallem auf das Problem der festen Vektorgröße des Kontextfensters eingeht [Bah16].

Bei der Übersetzung jedes einzelnen Wortes sucht das System im Ausgangssatz nach den Stellen mit den relevantesten Informationen. Dabei berücksichtigt es die Kontextvektoren der Wörter an diesen Positionen, sowie die der bereits übersetzten Wörter und generiert damit das nächste Wort [Bah16].

Der größte Unterschied zur originalen Encoder-Decoder Architektur ist, dass diese neue Architektur nicht versucht die gesamte Eingabesequenz in einen einzelnen Kontextvektor umzuwandeln, sondern die Eingabesequenz in eine Sequenz von Vektoren umzuwandelt. Dadurch können die Modelle besser mit längeren Sätzen umgehen [Bah16].

2.3 LLMs

Large Language Models sind eine Art der künstlichen Intelligenz, welche mithilfe probalistischer Methoden Text generiert. Heutzutage basieren Chatbots auf sogenannten Large Language Models. Diese Large Language Models sind Machine Learning Modelle, die auf der Transformer Architektur (siehe 2.3.1) aufbauen. Die Modelle werden mit einer sehr großen Menge an Daten trainiert und lernen anhand dieser ihre jeweilige Aufgabe. LLMs haben in verschiedenen Anwendungsgebieten einen nutzen. Heutzutage sind ein paar der bekanntesten Vertreter Chatbots, wie ChatGPT und Bildgeneratoren, wie Midjourney [Whaa].

2.3.1 Transformer

Die Transformer Architektur wurde 2017 von Vaswani et al in der Arbeit "Attention is all you need" vorgestellt. Vor dieser Architektur waren die besten Modelle Encoder-Decoder Architekturen (siehe 2.2.7), die mit dem Attention Mechanismus (siehe 2.2.8) ergänzt wurden und die auf Recurrent oder Convolutional Neuronalen Netzen aufgebaut waren.

Der Originale Transformer, der von Vaswani et al vorgestellt wurde, bestand ebenfalls aus einem Encoder und einem Decoder. Jedoch wurden keine Rekurrenten oder Convolutional Neuronalen Netze mehr genutzt. Die großen Neuheiten in dieser Architektur war das Konzept der Self-Attention, Parallelisierbarkeit, Multi-Head Attention und das weg lassen der Rekurrenten oder Convolutional Neuronalen Netze. [Vas23]

Self-Attention

Während beim ursprünglichen Attention Mechanismus (siehe 2.2.8) Beziehungen zwischen dem Encoder und dem Decoder modelliert wurden, ermöglicht Self-Attention es dem Modell, Beziehungen innerhalb einer einzelnen Sequenz zu erfassen.

Dies wird erreicht indem jedes Element einer Sequenz mit allen andere Elementen in Beziehung gesetzt wird. Dies hilft dabei den Kontext zu bestimmen.

Das heißt, durch die Self-Attention erkennt der Transfomer an welchen Position der Eingabe Sequenz er nachschauen muss um den aktuellen Vektor besser einbetten zu können [Vas23, Ala].

Bei Self-Attention wird jeder Eingabe-Vektor durch drei Vektoren repräsentiert. Einem Query Vektor $\mathbf{Q}$, einem Key Vektor $\mathbf{K}$ und einem Value Vektor $\mathbf{V}$. Diese Vektoren werden erstellt indem der eingebette Eingabe Vektor $\mathbf{X}$ mit jeweils einer der Gewichts Matrizen W^Q, W^K, W^V multipliziert wird.

$$Q = X\mathbf{W}^Q, K = X\mathbf{W}^K, V = X\mathbf{W}^V$$

Im nächsten Schritt werden die Skalarprodukte **Score** des Query Vektors des aktuell eingebetteten Vektors mit jeweils jedem Key Vektor der Token der aktuellen Eingabe berechnet.

$$\mathbf{Score_1} = Q_1 \cdot K_1$$

$$\mathbf{Score_2} = Q_1 \cdot K_2$$

$$...$$

$$\mathbf{Score_n} = Q_1 \cdot K_n$$

Anschließend wird dieses Skalarprodukt durch die Wurzel der Dimensionen des Key Vektors d_k geteilt und auf das Ergebnis wird eine Softmax Funktion angewendet, die diese Werte normalisiert und dafür sorgt, dass sie summiert 1 ergeben.

$$ScaledScore = softmax(\frac{Score_1}{\sqrt{d_k}})$$

Als nächstes werden die Softmax Ergebnisse mit den Value Vektoren multipliziert und diese gewichteten Werte werden aufsummiert.

$$Attention = \sum_{i=1}^{n} ScaledScore_i V_i$$

Dieser gesamt Prozess wird durch Matrix Berechnungen in einem Schritt erledigt mit der Formel:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Dies gibt uns den Attention Vektor für jeden Token [Vas23, Ala].

Multi-Head Attention

Multi-Head Attention wurde ebenfalls mit dem Self-Attention Mechanismus in der Transformer Architektur eingeführt. Durch diesen soll dem Transformer die Fähigkeit gegeben werden sich auf verschiedene Positionen zu konzentrieren, statt nur auf eine [Vas23, Ala].

Bei diesem Mechanismus werden in der Attention Layer statt eines einzigen mehrere Attention Heads verwendet. Im originalen Transformer wird mit 8 Attention Heads gearbeitet. Jeder dieser Heads führt den Self-Attention Mechanismus aus, aber jeder hat unterschiedliche Gewichts Matrizen

$$W^Q, W^K, W^V$$

Da jeder der Heads seine Berechnung mit unterschiedlichen Matrizen durchführt, können verschiedene Heads verschiedene Werte auf unterschiedliche Teile des Satzes legen. Durch diesen Mechanismus kann es zu einem besseren Verständnis für Nuancen kommen [Vas23, Ala].

Die finale Attention Matrix z, die dem jeweiligen Token zugeschrieben wird, wird über eine weitere Matrix berechnet. Hierfür werden die Ergebnisse der Attention Heads

$head_1, head_2, ..., head_n$ aneinander konkateniert und im Anschluss mit der Gewichts Matrix W_0 multipliziert [Vas23, Ala].

$$z = Concat(head_1, head_2, ..., head_n)W_0$$

Sliding-Window Attention

Sliding Window Attention ist eine Weiterentwicklung der Self-Attention. Hierbei werden nicht alle anderen Position betrachtet, sondern nur eine bestimmte Menge an Positionen, die um den aktuellen Token liegen. Dies hat den Vorteil, dass durch weniger betrachtete Positionen die benötigte Rechenleistung und Zeit sinkt. Dieser Ansatz ist zwar in der Lage in vielen Fällen, die notwendigen Kontexte zu erkennen, aber dies ist nicht in allen Fällen sicher. Dennoch ist dies durch die Vorteile ein Ansatz, der von einigen Unternehmen für ihre LLMs genutzt wird. [Bel20, Gem24]

Multi-Query Attention

Multi-Query Attention ist eine Weiterentwicklung der Multi-Head Attention. Dieser Mechanismus funktioniert ähnlich wie Multi-Head Attention, hierbei werden in dieser Variante jedoch die Key und Value Matrizen unter den verschiedenen Heads geteilt. Dadurch müssen diese großen Matrizen nicht öfter geladen werden und es wird an Speicher und an Zeit gespart. Dies sorgt vorallem für eine schnellere Inferenz. Hier gibt es auch nur eine kleine Verringerung der Qualität [Sha19].

Grouped-Query Attention

Die Grouped-Query Attention ist eine weitere Variante der Multi-Head Attention. Diese ist eine Variante, die von der Multi-Query Attention abstammt und zwischen Multi-Head Attention und Multi-Query Attention liegt. Hier wird ebenfalls die Anzahl der Key-Value Heads verändert. In dieser Version gibt es davon mehr als eine und weniger als die Anzahl der Query Heads. Inferenz funktioniert dadurch schneller als bei der Multi-Head Attention und es geht weniger Qualität verloren als bei Multi-Query Attention [Ain23].

Block Sparse Attention

Block Sparse Attention ist eine Kombination aus Sliding, Global und Random Verbindungen. Durch die Kombination der drei Verbindungen wird hierbei eine effiziente Balance zwischen den lokalen Details und dem globalen Kontext erreicht. Des weiteren reduziert dies die benötigte Rechenleistung [Gup].

2.3.2 Tokenizer

Tokenizer sind ein wichtiger Part der Machine Learning Toolchain im Zusammenhang mit dem Lernen von menschlicher Sprache. Wenn im Bereich des Natural Language Processing ein Machine Learning Modell trainiert werden soll, kommt die Frage auf, wie Texte aufgeteilt werden können, damit das Modell, diese effektiv verarbeiten kann.

Die Tokenizer teilen die Texte so auf, dass das zu trainierende Modell komplexe Sätze besser verarbeiten kann. Der Tokenizer macht das, indem er den Text in sogenannte Tokens aufteilt. Ein Token kann ein ganzes Wort, ein Teil eines Worts oder ein Satzzeichen sein.

Dies ist ein wichtiger Teil des NLP, da die Aufteilung der Trainingsdaten einen großen Einfluss darauf hat, wie gut das LLM funktioniert. Genau wie Compiler, die Programme in Tokens aufteilen, benutzen LLMs Tokenizer, damit die Texte aufgeteilt werden können und die dabei entstandenen Token über verschiedene grammatikalische Regeln zusammengesetzt werden können [Per21].

Eines der größten Probleme hierbei ist, dass verschiedene Sprachen verschiedene grammatikalische Regeln haben und somit verschiedene Arten des Zerlegens und Zusammensetzens brauchen. Ein weiteres Problem ist, dass das selbe Wort teils verschiedene Bedeutungen haben kann. Hierbei ist der Kontext und die Anzahl der Token, die das Modell erkennen kann wichtig. Es könnten alle möglichen Token aufgenommen werden, was aber für Overfitting innerhalb des Modells sorgen kann. Dazu kommt, dass das die notwendigen Kosten zum Trainieren und für die Inferenz des Systems erhöhen würde [Per21, Llaa].

2.3.3 Training

Das Training eines LLM hat zwei Schritte. Der erste Schritt ist das Pretraining, bei dem das LLM generell trainiert wird. Der zweite Schritt ist das Finetuning indem das vortrainierte Modell aus Schritt eins mit einem weiteren Datensatz trainiert wird und dadurch speziellere Kompetenzen erreicht, wie bessere Kenntnisse über eine bestimmte Domäne [Das24, Cra24].

Pretraining

Das Pretraining eines LLM setzt die Grundlage für die LLM. Für das Pretraining eines LLM werden sehr viele Daten benötigt und diese sollten eine hohe Qualität haben. Bessere Qualität dieser Daten, hilft dem LLM später bessere Ausgaben zu generieren. Daher ist es wichtig eine gute Datenpipeline für die Trainingsdaten vorzubereiten, welche die Datenqualität sicherstellen. [Cra24, Intc]

Um ein etwas besseres Bild davon zu haben, wie viele Daten benötigt werden, wird Llama3 von Meta betrachtet. Llama3 wurde mit 15 Billionen Token trainiert [Intc].

Um einen Vergleich zu haben, wird der fineweb Datensatz, welcher auf Huggingface zur Verfügung steht, betrachtet. Dieser Datensatz hat ebenfalls 15T Token und hat eine Speichergröße von 47.6TB [Pen24].

Für das Pretraining werden Daten aus den Bereichen benötigt, in denen die LLM später Dinge generieren soll. Ebenfalls die Sprache der Daten ist wichtig. Das Verhältnis der Menge der Daten zueinander beeinflusst ebenfalls die Performance der LLM.

Das Training ist der rechnerisch aufwändigste Teil der LLM Erstellung. Für das Training des Llama3 8B Parameter Models hat Meta 1.3 Millionen GPU Stunden gebraucht. Mit dem GPU Cluster, den Meta zur Vefügung hat, mit 24,576 H100 CPUs, hat es ca 2.2 Tage gedauert dies zu trainieren.[Llab, Intc, Met24]

In diesem Kontext gibt es auch verschiedene Unternehmen die Partnerschaften mit KI Unternehmen schließen und diesen Zugriff auf ihre Daten geben, damit sie besser in diesem Bereich werden. Ein Beispiel für so einen Zusammenschluss ist der Zusammenschluss von OpenAI und Stackoverflow, wobei OpenAI hofft, dadurch ein Model mit besseren Coding Kompetenzen zu erhalten und Stackoverflow hofft, dass ihre Nutzer dadurch mehr mit Inhalt auf der Seite interagieren [Sta].

Finetuning

Beim Finetuning einer LLM wird auf dem vortrainierten Modell aufgebaut und das Modell wird mit einem kleineren spezifischeren Datensatz weiter trainiert. Dieser Schritt ist rechnerisch weniger aufwändig als das vortrainieren und ist daher für mehr Unternehmen und Instiutionen zugänglich. Hinzu kommt, dass dieser Schritt weniger zeitaufwändig ist [Das24].

Um einen Vergleich für den Aufwand zu haben wird `DiscoResearch/Llama3-German-8B` auf Huggingface betrachtet. Das Model, welches 1.3 Millionen GPU Stunden für das Pretraining gekostet hat, wird jetzt gefinetuned. DiscoResearch hat hierfür Zugriff auf den Supercomputer 42 von hessian.Ai bekommen. Das Model wurde von 128 GPUs innerhalb von ca 60 Stunden über 65 Milliarden Tokens gefinetuned. Dies heißt, es hat ca 7680 GPU Stunden gekostet [Dis24a].

Teilweise wird Finetuning verwendet um sogenannte Instruct Modelle zu erstellen. Diese Instruct Modelle sind Versionen des Standard Models, die darauf finetuned wurden, Instruktion zu befolgen. Bei einem Model, wie Llama3 von Meta, ist die nicht finetuned Variante des Models bereits in der Lage Instruktionen zu befolgen, wird aber durch ein kurzes Finetuning messbar besser in diesem Bereich [Tou].

Eine weitere Richtung in die Finetuning gehen kann ist das Trainieren eines Models auf einer neuen Sprache. Hierfür wird erneut `DiscoResearch/Llama3-German-8B` auf betrachtet. Dies ist eine auf deutsch finetuned Version von Llama3. Sie wurde mit Benchmarks veröffentlicht, die zeigen, dass die deutsche Leistung des Models sich durch das finetuning über 65 Milliarden Token verbessert hat. Die Englische Leistung ist etwas verschlechtert, aber vorallem beim hellaswag_de [Leo] Benchmark, ist die Leistung messbar besser. [Dis24a]

2.3.4 Inferenz & Generation

Sobald ein Model trainiert wurde und bereit ist benutzt zu werden, muss es ausgeführt werden. Dieser Prozess nennt sich Inferenz. Um Inferenz ausführen zu können gibt es verschiedene Möglichkeiten. Es gibt die Möglichkeit Inferenz über online APIs wie beispielsweise die Huggingface Inference API [Ser] oder ähnliche Services auszuführen. Eine weitere Möglichkeit ist es die Modelle lokal auszuführen.

Inference

Wenn eine der APIs genutzt wird um mit einem Modell zu arbeiten, können Anfragen an die API geschickt werden und das Ausführen des Models wird von den Betreibern der API übernommen.

Die andere Option, das Model lokal auszuführen, kann über verschiedene Methoden passieren. Eine Option ist es einen Service wie Ollama (siehe 2.8) oder LM Studio nutzen [LMS], worüber die Modelle gedownloadet und ausgeführt werden können. Mit Ollama kann mit dem Modell über die CLI interagiert werden. Über LM Studio kann mittels einer GUI mit dem Modell interagiert werden. Beide Projekte basieren auf llama.cpp (siehe 2.7).

Eine andere Option, das Model lokal auszuführen, ist es die Gewichte der Modelle zu downloaden und diese mittels Code auszuführen. Hierfür können die Modelle teils direkt bei den Unternehmen, wie bei Llama von Meta, gedownloadt werden. Eine Website auf der viele Modelle veröffentlicht werden ist Huggingface.co (siehe 2.10).

Andrej Karpathy [Kar] hat gezeigt, dass mit einem 15 Millionen Parameter Model von Llama2 in C ohne externe Bibliotheken in unter 1000 Zeilen Inferenz betrieben werden kann [Llaa]. Modelle die heute veröffentlicht werden haben weit über 15 Millionen Parameter, daher ist die Inferenz mit so einem Programm nicht sehr effektiv. Das Llama2 Model mit 7 Milliarden Parametern generiert auf einem M1 Macbook Air mit diesem Projekt einen Token pro 30 Sekunden. Modelle in einer Milliarden Größe, brauchen aufgrund der Größe ein paar Optimierungen um nutzbar zu sein [Llaa].

Eine Möglichkeit Modelle schneller zu machen ist es sie zu quantisieren, dieser Prozess wird in 2.7 genauer erklärt.

Statt den Code für die Inferenz selbst zu schreiben gibt es auch noch die Möglichkeit verschiedene Frameworks zu verwenden. Drei dieser Frameworks werden in 2.9 vorgestellt und erklärt.

Generation

Die Generation der LLMs kommt auf ihre Trainingsdaten an. Genau wie beim Training arbeiten die LLMs beim generieren ihrer Ausgaben in Tokens. Hierfür nutzen die Modelle ihre vorher durch Training antrainierten Parameter dazu den nächsten Token hervorzusagen.

Diese Vorhersage findet sequentiell statt und nutzt dabei die neu generierten Tokens bei der Vorhersage des nächsten Tokens. Hierbei werden alle bisherigen Tokens für die Generierung des nächsten Tokens berücksichtigt, bis das maximale Kontextfenster, welches in 2.3.4 beschrieben wird, erreicht wurde [Gen].

Kontextfenster

Als Kontextfenster werden die Menge an Tokens bezeichnet, die ein ein Modell maximal zum Zeitpunkt der Generierung betrachten kann. Die Token außerhalb des Kontextfensters, werden bei der Generation nicht mehr berücksichtigt. Das Kontextfenster ist die maximale Eingabesequenzlänge des Transformers [Koh].

2.3.5 Limitationen & bekannte Probleme

LLMs haben trotz ihrer ständigen Verbesserungen einige Probleme, die schon seit einige LLM Generationen bekannt sind.

Ein Problem, welches LLMs haben ist, dass die Informationen, die LLMs haben, teils veraltet ist. Dies ist darauf zurückzuführen, dass die LLMs zwar mit sehr vielen Datensätzen trainiert werden, welche aber zeitlich, bis zum Zeitpunkt des Trainings, begrenzt sind. Der Trainingsprozess ist zeit- und ressourcenintensiv, während dem Training und dem Finetuning des Modells entsteht eine Lücke zwischen dem Wissensstand des Modells und der realen Welt. Um dieses Problem zu lösen gibt es Ansätze, wie RAG, welches in 2.6 näher erklärt wird.

Ein weiteres Problem, ist die feste Größe des Kontextfensters der LLMs. Dies führt je nach Aufgabe dazu, dass Kontexte nicht beachtet werden oder Informationen, die zu weit zurück liegen, nicht beachten werden können [Wan24b].

Obwohl LLMs mit sehr vielen Datensätzen trainiert werden, ist ein Problem, dass LLMs halluzinieren können. Das heißt, das Modell generiert plausibel klingende Informationen, die aber faktisch nicht korrekt sind, entweder im Zusammenhang mit der Eingabe des Nutzers oder der zum Training genutzten Daten [Hua23].

Das letzte Problem, welches in dieser Arbeit besprochen wird ist der Reversal Curse. Dies ist ein Problem der LLMs im Zusammenhang mit Generalisierung von Sprache. Der Reversal Curse tritt auf, wenn ein Modell mit einem Satz der Form "A ist B" trainiert wird. Manchmal ist die LLM nicht in der Lage durch diesen Satz die umgekehrte Schlussfolgerung zu lernen. Hier ist ein bekannter Fall in GPT-4 in dem GPT-4 die Frage "Wer ist die Mutter von Tom Cruise?" in 79% der Fälle richtig mit "Mary Lee Pfeiffer" beantwortet und die Frage "Wer ist der Sohn von Mary Lee Pfeiffer?" nur in 33% der Fälle richtig beantwortet [Ber24].

2.3.6 Prompting

Ein Prompt im Bereich der LLMs ist die Texteingabe des Nutzers, welche die Ausgabe des LLMs bestimmt. Die Art und Weise der Eingabe kann von LLM zu LLM unterschiedlich sein. Bei Modellen, welche Bilder generieren, wie Midjourney oder DALLE-3 sind die Prompts eher beschreibend und bei Prompts für Chatbots, wie ChatGPT sind die Prompts eher Fragestellungen oder Anfragen [Ama24].

Die Prompts können starken Einfluss auf die Qualität der Ausgaben des Systems haben. Es gibt verschiedene Methoden, die mit der Zeit gefunden wurden um die Ausgaben des Systems zu verbessern. Aufgrund dessen hat sich im Bereich der LLMs eine neue Disziplin namens Prompt Engineering gebildet. Innerhalb dieser geht es darum, Prompts so zu optimieren, dass das LLM System genau das macht was der Nutzer will [Ama24].

Egal wie gut ein Prompt ist, manchmal kann eine LLM Applikation die gewollte Ausgabe nicht ausgeben. Dies kann verschiedene Gründe haben, wie zum Beispiel fehlende Informationen. Die Informationen können fehlen, weil sie nicht Teil des Trainingsdatensatz der LLM waren. Aufgrund dessen ist es wichtig, die Limitationen der LLM, zu kennen [Ama24].

Es gibt zwar keinen eindeutigen Weg gute Prompts zu generieren, aber es gibt einige Best Practices.

Im Prompt nur die Aufgaben mitteilen nennt sich Zero-Shot Prompting. Das heißt es werden keine Beispiele mitgegeben und manche LLMs sind in der Lage Zero-Shot Prompts in manchen Aufgabenbereichen zu lösen. Der LLM im Prompt ein Beispiel für die gewollte Ausgabe mitzugeben nennt sich One-Shot Prompting und statt nur einem Beispiel, mehrere mitgegeben heißt few-shot prompting. Die Qualität der gewollten Ausgaben lässt sich teils mithilfe von One-Shot oder Few-Shot Prompts statt Zero-Shot Prompts verbessern. Jedoch hat der Input Kontext eine begrenzte Größe und umso größer der Input ist, umso größer ist der rechnerisch benötigte Aufwand, daher ist das eine Lösung, die nicht alle Probleme beseitigen kann [Ama24]. Ein paar weitere Tipps sind:

- Anweisungen an den Anfang oder das Ende des Prompts stellen.
- Spezifische Anweisungen und genaue Beschreibungen geben. Auch spezifisch und genau bei den gewollten Ausgaben sein.
- Anweisungen die der LLM mitteilen was sie tun soll, statt den Dingen die sie nicht tun soll, sind zu bevorzugen.

Es gibt auch ein paar fortgeschrittene Methoden die verwendet werden können [Ama24].

Hierzu gehören:

- Der LLM durchsetzungsstark mitteilen, was von ihr verlangt wird. Hierbei kann auch in Großbuchstaben und mit Ausrufezeichen gearbeitet werden.

- Gedankengang Prompting, bei dem die LLM, die einzelnen Schritte die sie gemacht hat und die Gründe dafür erklären soll.

Es gibt eine spezielle Art der Prompts namens System Prompt. Dieses System Prompt dient dazu das Verhalten der LLM in einem Rahmen zu steuern. Durch dieses System Prompt kann der LLM mitgeteilt werden, welche Rolle sie einnimmt und unter anderem auch ihren Stil beeinflussen [Giv].

2.4 Vektor Datenbanken

Vektor Datenkbanken sind Datenbanken, die hochdimensionale Vektoren speichern können. Diese Vektoren sind mathematische Repräsentationen von Attributen oder Merkmalen. Je nachdem wie granular die Daten, die abgespeichert werden sollen sind, kann es sein, dass die Daten tausende Dimensionen haben. Die Rohdaten werden mittels Einbettungsfunktion oder Transformationen in ihre Vektordarstellung umgewandelt und dann abgespeichert [Han23].

Diese Art der Datenbanken hat für den KI und Machine Learning Bereich ein paar Vorteile im Vergleich zu traditionellen Datenbanken.

Ähnlichkeitssuche: Traditionell Datenbanken brauchen klar definierte Kriterien für die Suche. Hier ist es nicht möglich eine Ähnlichkeitssuche, wie bei Vektordatenbanken durchzuführen. Bei Vektordatenbanken können mittels Ähnlichkeitssuche-Verfahren schnell ähnliche Datensätze gefunden und ausgeben werden [Han23].

Support für unstrukturierte und komplexe Daten: Traditionelle Datenbanken brauchen strukturierte Daten. Die Daten die für Machine Learning genutzt werden sind oft unstrukturierte Daten, die verschieden Formate haben oder komplett verschiedene Datentypen sind, dies schließt auch Audio und Videodateien mit ein. Vektordatenbanken können diese Dateien durch die Vektordarstellung abspeichern und sind damit besser für diesen Fall geeignet [Han23].

Skalierbarkeit und Performance: Dadurch, dass Vektordatenbanken oft mit Big Data verwendet werden, gibt es einige Ansätze zum Verbessern der Performance und Skalierbarkeit. Ein paar der Methoden die verwendet werden sind Partitioning, Sharding und Caching [Han23].

2.5 Embedding Modelle

Embedding Modelle dienen dazu Daten in eine Vektorrepräsentation umzuwandeln. Diese Modelle sind in der Lage verschiedene Datenformate umzuwandeln. Unter anderem Text, Video und auch Bilder.

Diese Vektoren sind hochdiemnsional und sollen dadurch auch Nuancen und die Bedeutung der Wörter mit in die Vektorrepräsentation aufnehmen. Dadurch, dass der Text in die jeweilige Vektorrepräsentation umgewandelt wird, können nach der Umwandlung verschiedene mathematische Verfahren auf diesen Vektor angewendet werden. Dazu zählen Verfahren wie Semantische Suche und Semantische Nähe. Dies sorgt ebenfalls dafür, dass das Arbeiten mit den Daten schneller ist [Lee24, Mik13, NVI].

Embedding Modelle werden in LLM Applikationen dafür benutzt, Daten in ihre Vektorrepräsentation umzuwandeln und diese Vektoren zusammen mit den dazugehörigen Daten in einer Vektordatenbank zu speichern. [Lee24, Mik13, NVI]

Semantische Suche

Bei der semantischen Suche handelt es sich um ein Verfahren, welches dafür genutzt wird Daten semantisch zu vergleichen. Die Daten werden hierfür in ihre Vektordarstellung umgewandelt und dann werden die Vektoren miteinander verglichen und die top k Vektoren mit der höchsten Semantischen Nähe werden zurückgegeben [NVI, Cha22a, Whad].

Hierfür gibt es verschiedene Verfahren, drei Verfahren, die oft verwendet werden sind:

- Euklidischer Abstand
$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2} \tag{2.1}$$
- Kosinus-Ähnlichkeit
$$\cos(\theta) = \frac{\sum_{i=1}^{n} p_i q_i}{\sqrt{\sum_{i=1}^{n} p_i^2} \sqrt{\sum_{i=1}^{n} q_i^2}} \tag{2.2}$$
- Manhattan-Metrik
$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i| \tag{2.3}$$

Mittels einer dieser oder eines anderen Verfahrens wird dann die Semantische Nähe berechnet und es werden die passenden Dateien, falls es diese gibt, zurückgegeben. Dies hilft den LLM Applikationen dabei, passende Daten schnell abzurufen [NVI, Cha22a, Whad].

2.6 RAG

Eines der größten Probleme der LLMs heutzutage ist, dass die Informationen, die sie zum beantworten für Fragen benutzen, teilweise veraltet oder sehr allgemein sind. Eine Technologie, die versucht, dieses Problem zu beseitigen, ist die sogenannte Retrieval Augmented Generation. Die Retrieval Augmented Generation soll das parametrische Gedächtnis der LLM mittels eines nicht parametrischen Gedächtnisses erweitern [Lew21].

Retrieval Augmented Generation löst ein paar Probleme vor denen LLMs heutzutage stehen, ist aber keine perfekte Lösung und hat selbst einige Probleme, die gelöst werden müssen. In diesem Zusammenhang kann RAG in verschiedene Stufen eingeordnet werden.[Gao24]

2.6.1 Naive RAG

Naive RAG ist die traditionelle Herangehensweise an das Problem der RAG. Sie besteht aus drei Stufen. Indexing, Abfrage und Generierung.

Beim Indizieren werden die rohen Eingabedaten gereinigt und extrahiert. Dies ist unter anderem wichtig, da die Daten verschiedene Formate haben können, wie PDF, Markdown oder HTML. Diese Texte werden nach der Reinigung in Plain Text umgewandelt und dieser Plain Text wird dann in kleinere Teile unterteilt. Diese kleineren Teile werden über ein Embedding Modell in eine Vektordarstellung umgewandelt und dann in einer Vektordatenbank abgespeichert.[Gao24]

Bei der Abfrage erhält das RAG System eine Anfrage und benutzt das selbe Embedding Modell, wie in der Indizierungsphase, um die Abfrage in eine Vektordarstellung umzuwandeln. Im Anschluss wird durch einen Semantischen Vergleich nach den Top-k Ergebnissen gesucht und diese werden ausgegeben.[Gao24]

Bei der Generation bekommt die LLM dann die Abfrage und die abgerufenen Ergebnisse in einem Prompt zurück und generiert daraus eine Antwort. Was genau hier beachtet wird kommt auf die LLM, das Kontext Fenster und den benutzten Prompt an[Gao24].

Wie in 2.6 bereits erwähnt gibt es hierbei ein paar Probleme. Das Hauptproblem, welches hier besteht ist die der Abfrage. Bei der Abfrage gibt es oft Probleme, welche mit der Genauigkeit zu tun haben. Dementsprechend kann es passieren, dass das RAG System falsche oder unpassende Informationen zurück gibt. Aufgrund der Probleme bei der Abfrage entstehen dann ebenfalls Probleme bei der Generierung der Antwort. Was dazu führt, dass die Qualität der Antworten sinkt. Dadurch sinkt dann ebenfalls die Verlässlichkeit auf das System[Gao24].

2.6.2 Advanced RAG

Advanced RAG wurde entwickelt um den Problemen der naiven RAG entgegenzuwirken. Entsprechend wurden hier einige Änderungen vorgenommen, insbesondere bei der Abfrage. Um die Abfrage zu verbessern wurden dieser pre- und post Strategien hinzugefügt. Dazu wurde der Indexing Prozess angepasst [Gao24, Lew21]. Beim Indexing wird nun Metadata mit eingebunden und der Text wird genauer unterteilt.

Pre-Abfrage

Bei der pre-Abfrage wird vorallem darauf geachtet die Anfrage so klar wie möglich zu machen. Hierfür werden Metadaten angehängt, Anfragen umgewandelt, Anfragen erweitert und noch einige weitere Dinge abgeändert. Das Ziel hier ist es die Anfrage zu ändern, sodass sie so klar wie möglich ist und sie für Retrival zu optimieren [Gao24, Lew21].

Post-Abfrage

Nach der Abfrage der relevanten Daten, wird versucht diese möglichst optimal mit der Anfrage zu verbinden. Hierbei liegt der Fokus darauf diese zu komprimieren und sie neu Anzuordnen. Die wichtigsten Dokumente werden an eines der Enden der Anfrage geschoben und es wird versucht nur die wichtigsten Informationen heraus zu suchen und diese zurück zu geben. Dadurch sollte die LLM genug Informationen erlangen, aber nicht zu viele und die Antworten sollten besser werden[Gao24, Lew21].

2.7 llama.cpp

Ein Problem, welches im Abschnitt 2.3.4 bereits angesprochen wurde, ist, dass es schwierig sein kann, für Personen oder Unternehmen, die nicht über die nötige Hardware verfügen, LLM Inferenz zu betreiben. Desto größer die LLM, die benutzt werden soll, wird, desto aufwändiger werden die Anforderungen an die Hardware. Ein großes Open Source Projekt, welches sich mit diesem Problem auseinander setzt ist llama.cpp.

Dieses Projekt hat zu Beginn viele Nutzer angezogen, da durch dieses Projekt, ohne spezielle Hardware, Inferenz betrieben werden konnte.

Zu Beginn wurde das Projekt hauptsächlich auf Inferenz auf der CPU ausgelegt. Dies gab den Nutzern zwar die Möglichkeit auf ihren Maschinen ohne GPU Inferenz zu betreiben, aber durch fehlende GPU Einbindung hatte dies Begrenzungen in der Geschwindigkeit und der Größe der Modelle, die benutzt werden konnten.

Mit der Zeit wurde dem Projekt auch die Möglichkeit hinzugefügt, GPUs für die Inferenz zu benutzen, wodurch die Nutzer in der Lage sind schnellere Inferenz mit größeren Modellen auszuführen.

Damit die verschiedenen Modelle über llama.cpp nutzbar sind, gab es hier zu Beginn das GGML format. Dies wurde ebenfalls von Gerganov im Zusammenhang mit seinen Projekten entwickelt.Ein Vorteil hierbei war, dass das GGML Format das Speichern von quantisierten Modellen erlaubt. Quantisierte Modelle haben hierbei den Vorteil, dass die Präzision der einzelnen Parameter von bspw. 32 Bit Gleitkommazahlen auf eine niedrigere Genauigkeit, wie bspw. 8 Bit Gleitkommazahlen konvertiert wurde. Der Vorteil davon war, dass die Inferenz der einzelnen Modelle weniger intensive Rechenleistung erfordert hat und durch die kleinere Größe auch weniger Speicher benötigte. Dadurch war es einfacher möglich, die Inferenz auf der CPU durchzuführen.

Jedoch hatte diese Format einige Probleme. Zum einen hat jede Architektur ihr eigenes Skript benötigt, damit sie in das GGML Format konvertiert werden kann, dazu kam, dass in der Datei nicht erkennbar war, für welche Architektur das Modell war. Heutzutage benutzt llama.cpp ein neues Format, namens GGUF, welche einige der Probleme von GGML beseitigen soll. Ein großer Vorteil von diesem Format ist, dass das Modell nun alle Informationen die zum Laden des Modells notwendig sind beinhaltet und es müssen keine zusätzlichen Informationen mehr vom Nutzer gegeben werden. Unter anderem beinhaltet die Datei nun die Architektur des Modells [Ger24a, Ger24b].

2.8 Ollama

Ollama ist ein Open Source Projekt, welches seinen Nutzern dabei helfen soll, LLMs schneller auf ihren lokalen Maschinen zu starten, mit diesen zu interagieren und die Generation der LLMs zu beschleunigen. Das Backend das Ollama unterstützt ist llama.cpp und es soll das Downloaden, Installieren und das Interagieren mit den LLMs einfacher machen.

Ollama hat seine eigene Modellbibliothek über die Nutzer LLM Modelle downloaden können und diese dann direkt nutzen. Wenn ein Modell nicht in der Bibliothek zu finden ist, können Modelle mittels Modelfile importiert werden. Dies kann mit Modellen im GGUF Format, welches in 2.7 genauer erklärt wurde, und mit sogenannten Safetensor Dateien gemacht werden [Oll24]. Das Safetensor Format wurde von Huggingface eingeführt um die Unsicherheiten des vorher genutzten Pickle Formats zu beseitigen. Eines der größten Probleme dieses Formats war, dass beim Deserialisieren eines Pickle Objekts schadhafter Code ausgeführt werden konnte, da Pickle den Unterschied zwischen schadhaften und sicherem Code nicht erkennen kann.[Saf] Mit dem neuen Safetensor Format konnte vorallem dieses Problem beseitigt werden.

Ollama kann Safetensor Dateien für die drei Architekturen LlamaForCausalLM, MistralForCausalLM und GemmaForCausalLM ausführen. Wenn ein Modell genutzt werden soll, welches weder das GGUF Format noch das Safetensor Format für eine der drei genannten Architekturen hat, muss dieses Modell erst in GGUF umgewandelt werden. Dafür kann das convert script aus dem llama.cpp Github Projekt genutzt werden.

Im Anschluss kann das Modell mithilfe einer Modelfile importiert werden. Innerhalb dieser Modelfile können noch weitere Dinge festgelegt werden, wie das System Prompt das genutzt werden soll [Oll24].

2.9 KI Frameworks

Dadurch, dass LLMs nun ein größeres Thema im Auge der Öffentlichkeit sind und es durch Bibliotheken wie llama.cpp einfacher geworden ist, für Personen, ohne besondere Hardware, LLMs zu nutzen, gibt es immer mehr Interesse an Frameworks, mit denen LLM basierte Applikationen entwickelt werden können. Dieses Interesse kann über Github Stars, die in A.4 dargestellt sind, beobachtet werden.

Diese Frameworks, dienen dazu das Arbeiten mit diesen Modellen zu vereinfachen. Ein großer Fokus bei den meisten davon liegt darauf den Nutzern das Arbeiten mit eigenen Daten einfacher zu machen, indem beispielweise eine RAG Pipeline genutzt wird. Hier werden einige der größten Frameworks vorgestellt.

2.9.1 Haystack

Haystack ist ein Open Source Framework, welches LLM Applikationen erstellen kann. Der Fokus hier liegt vorallem auf RAG Applikationen und Such Systemen, welche mit großen Mengen an Daten arbeiten können [Whab].

Bei Open Source Frameworks ist die Community einer der wichtigsten Faktoren. Umso mehr Teilnehmer an einem Open Source Projekt es gibt, umso mehr Unterstützung gibt es und umso mehr Fehler fallen den Leuten auf, die dann beseitigt werden können. Haystack hat 14.3k Sterne auf Github, zum Zeitpunkt dieser Arbeit. Das macht es zum kleinsten Projekt auf dieser Liste [Hay].

Haystack kann direkt Modelle von Huggingface, OpenAI und anderen Quellen beziehen und diese ausführen. Dazu können Modelle genutzt werden, die auf Hosting Plattformen, wie Azure oder Sagemaker gehostet werden [Whab].

Die zwei Konzepte die hierbei am wichtigsten sind, sind Komponenten und Pipelines. Komponenten sind Blöcke, welche Aufgaben durchführen können. Aufgaben, die eine Komponente ausführen kann, sind zum Beispiel Text Generation und das Erzeugen von Einbettungen [Whab].

Eine Pipeline besteht aus verschiedenen Komponenten. Durch das Zusammensetzen von Komponenten zu einer Pipeline kann der Datenfluss innerhalb der Applikation genau beeinflusst werden [Whab].

Die Programmiersprache für Haystack ist Python[Whab, Hay].

2.9.2 Langchain

Langchain ist ein weiteres Open Source Framework, welches für das Erstellen von LLM Applikationen erstellt wurde. Im Gegensatz zu Haystack ist Langchain nicht nur auf das Erstellen von RAG Applikationen und Such Systemen fokussiert. Langhchain versucht möglichst vielseitig einsetzbar zu sein [Lana].

Langchain ist, in Hinsicht auf die Github Sterne das größte Framwork, welches hier vorgestellt wird. Zur Zeitpunkt des Schreibens dieser Arbeit, hat das Projekt 87.6k Sterne auf Github [Cha22b].

Für die Entwicklung liegt der Fokus bei Langchain auf Chains. Welche ähnlich wie Pipelines den Datenfluss bestimmen. Hierfür werden die einzelnen Teile, wie ein Retriever, der mit einer Vektor Datenbank verbunden ist, und ein LLM, erstellt. Diese Teile werden im Anschluss zu einer Chain verbunden. Mit dieser kann interagiert werden oder sie kann an

einer anderen Stelle eingebaut werden kann. Eine Möglichkeit wäre es diese per RestAPI verfügbar zu machen [Lanb].

Dazu gibt es das Paket Langserve, welches darauf ausgelegt ist, Chains als RestAPIs zu veröffentlichen. Dazu kommt ein Paket namens Langsmith, welches zum Monitoring und zum Evaluieren von LLM Applikation dient. Langsmith ist nicht Open Source und kostet für Teams oder für Hobby Entwickler ab einer gewissen Menge an Nutzungen ein monatliches Abbonement [Land, Lanc].

Langchain ist ein Framwork, welches für Javascript und für Python zur Verfügung steht. [Cha22b, Intd]

2.9.3 LlamaIndex

LlamaIndex ist ein weiteres Open Source Framework, das entwickelt wurde, um die Erstellung von LLM Applikationen zu erleichtern.

Zum Zeitpunkt des Schreibens dieser Arbeit hat das Projekt 32.8k Sterne auf GitHub und ist damit das zweitgrößte Projekt, welches hier vorgestellt wird [Liu22].

Bei LlamaIndex liegt der Fokus auf dem Arbeiten mit Daten. Deshalb ist einer der Fokus Punkte des Frameworks der Index. RAG Use Cases in LlamaIndex benutzen in ihrem Kern den Index als Kernkonzept. Indexe werden aus Dokumenten erstellt und sind im Grunde Knoten-Objekte, welche Teile von den Originaldokumenten respräsentieren [Ind, Llad]

LlamaIndex hat eine Plattform namens LlamaCloud, welche einen durch generative künstliche Intelligenz angetriebenen Parser hat. Dieses Projekt heißt LlamaParse und lässt sich mit LlamaIndex integrieren. LlamaParse kann ohne die LlamaCloud genutzt werden. Wenn die LlamaCloud genutzt wird können die für RAG verwendeten Daten dort über eine API gespeichert werden. Über die API können direkt verschiedenen Datenquellen angebunden werden [Llac, Intb].

LlamaIndex ist ein Framework, welches, wie Langchain, für Javascript und für Python zur Verfügung steht[Liu22].

2.10 Huggingface & MTEB

Huggingface ist eine Plattform, auf der verschiedene Daten gehostet werden, Diskussionen stattfinden und KI Apps zu finden sind [Hug24].

Auf dieser Plattform werden Daten in Git basierten Repositories gelagert. Zu den Daten die hier zu finden sind, gehören unter anderem Open Source LLM Modelle, Open Source Embedding Modelle und Datensätze zum trainieren von solchen Modellen. Es gibt die Möglichkeit KI-basierte Demo Apps in sogenannten Spaces auszuprobieren oder zu hosten. Hinzu kommt die Möglichkeit, ähnlich wie auf Social Media Plattformen, Beiträge zu veröffentlichen [Hug24].

Neben den Demo Apps die in Spaces zu finden sind, werden auch Ranglisten veröffentlicht. Eine der größten Ranglisten ist die MTEB Rangliste. MTEB steht für Massive Text Embedding Benchmark. Diese Rangliste wurde für Einbettungsmodelle erstellt und wird regelmäßig geupdated. Die Modelle werden hier in 8 verschiedenen Aufgabenbereichen verglichen [Mue23, MTE].

Ein paar der Aufgabenbereiche sind Clustering, Semantische Nähe und Zusammenfassungen. Die Datensätze wurden so ausgewählt, dass es eine verschiedene Länge an Texten gibt, damit verglichen werden kann, wie die Modelle mit kurzen und auch mit langen Texten

umgehen können. Aktuell ist das beste Modell im Durchschnitt in dieser Rangliste das Modell “SFR-Embedding-2_R” von Salesforce Research [Mue23, MTE].

2.11 Google Chat

Google Chat ist eine von Google entwickelte Chatplattform. Diese Plattform hat 2017 Google Hangout zusammen mit Google Meet ersetzt. Zu Beginn war das Projekt nur im Zusammenhang mit Google Workspace erhältlich und somit war es eine Chatplattform die hauptsächlich für Teams und Unternehmen nutzbar war. 2021 wurde die Plattform im Early Access auch für normale Benutzer erreichbar gemacht und ist seit 2022 für jeden benutzbar [Goo24].

Diese Plattform ist die Plattform, die die Techdivision hauptsächlich intern zur Kommunikation benutzt. Es gibt auf der einen Seite 1 zu 1 Chats in der eine Person mit nur einer anderer Person kommuniziert und auf der anderen Seite gibt es Innerhalb dieses Services die Möglichkeit sogenannte Spaces zu erstellen, welche Gruppenchats ähneln. In einen solchen Space, soll der Chatbot final eingebaut werden.

3 Anforderungsanalyse

Techdivision

Die Techdivision GmbH ist ein IT-Dienstleister aus Kolbermoor. Sie bietet Leistungen in vier Kernbereichen.

1. Consulting
2. Software & Solutions
3. Daten & Prozesse
4. Marketing & Kreation

Über diese vier Bereiche möchte die Firma den Kunden bei der digitalen Transformation helfen [Uns].

Innerhalb jedem dieser Bereiche gibt es verschiedene Teams die sich um spezifische Aufgaben des Teilbereichs kümmern.

Die Mitarbeiter der Techdivision kommunizieren, wenn nicht in Person, über den Google Chat. Die einzelnen Fachbereiche haben eigene Spaces innerhalb des Google Chats. Einer dieser Chats ist der IT-Support Chat, in dem Mitarbeiter technische Probleme melden oder bei technischen Problemen Fragen stellen können und für diese Unterstützung vom IT-Support erhalten. Für diesen soll der Chatbot entwickelt werden.

3.1 Ausgangslage

Die Ausgangslage des IT Supports innerhalb der Techdivision ist, dass der IT-Support hauptsächlich über den internen Google Chat oder über IT-Helpdesk Tickets betrieben wird. Die Tickets werden, wenn es Probleme sind, welche keine akute Hilfe benötigen, verwendet. Probleme, welche akute Hilfe benötigen, sind zeitkritische und blockende Probleme. Blockende Probleme sind Probleme, durch die nicht weiter gearbeitet werden kann.

Für akute Probleme ist der IT-Support Chat zuständig. Dies ist eine offene Chatgruppe für Mitarbeiter der Techdivsion und innerhalb dieser können Fragen gestellt werden, die von einem der Mitarbeiter des IT-Support Teams beantwortet werden. Falls es Fragen sind, welche nicht über den Chat gelöst werden können, dann wird ein Google Meet Meeting aufgesetzt oder, falls der jeweilige Mitarbeiter sich im Büro befindet, dann kann das auch vor Ort stattfinden.

Informationen und How To's der Techdivsion sind in den verschiedenen Datenquellen der Techdivsion zu finden.

Hierfür gibt es ein Confluence System. Hier werden die Daten nach Bereichen geordnet, wobei verschiedene Teams verschiedene Bereiche haben. Der Bereich, der für diese Arbeit am relevantesten ist ist der DevOps Bereich. In diesem werden verschiedene Technologien erklärt und mittels Anleitungen wird erklärt, wie diese eingerichtet werden können. Dazu werden

typische Probleme und deren Lösung genannt. Dazu gibt es ein internes Stackoverflow, in dem die Mitarbeiter Fragen stellen können und andere Mitarbeiter können diese beantworten oder kommentieren. Dazu gibt es verschiedene Dateien in Google Drive, die abgespeichert sind.

3.2 Anforderungen

Nun wird gefordert, dass dem IT-Support Gruppenchat ein Chatbot hinzugefügt wird, der den Mitarbeitern des Support Teams helfen soll, indem er einfache Fragen, die mithilfe von firmeninterner Dokumentation beantwortet werden kann, beantworten soll.

Im Folgenden werden die Anforderungen, die der Chat erfüllen soll vorgestellt. Diese Anforderungen wurde innerhalb von Meetings aufgestellt und anschließend ausgearbeitet.

3.2.1 A1 Selbst Hosting

Das größte Thema, welches beim Betreiben des Chatbots beachtet werden soll, ist der Datenschutz. Datenschutz ist ein großes Thema bei Unternehmen und vor allem, bei Unternehmen die mit Kundendaten arbeiten müssen oder auch bei internen Betriebsgeheimnissen, die nicht an die Öffentlichkeit weiter gegeben werden dürfen.

Die Techdivision ist ein IT-Dienstleister und somit ist dies auch ein Bereich der bei der Konzeption und Entwicklung dieses Chatbots beachtet werden muss. Aufgrund dieser Sorgen wird gefordert, dass der Chatbot von der Techdivision gehostet wird, was bedeutet, dass keine APIs von externen LLM-Systemen hierfür verwendet werden, die von anderen Unternehmen gehostet werden, wie beispielsweise die ChatGPT API.

3.2.2 A2 Integration in Google Chat

Das Hauptkommunikationsmittel, welches innerhalb der Techdivsion genutzt wird ist Google Chat. Teilweise findet Kommunikation auch über Email statt, jedoch ist dies nur ein kleiner Teil der Kommunikation.

Dazu kommt, dass der IT-Support innerhalb der Techdivsion hautpsächlich im Google Chat stattfindet

3.2.3 A3 Gruppenchat-Funktionalität

Auch, wenn die LLMs heutzutage weit entwickelt sind, können diese falsche Informationen weiter geben, oder komplett neue halluzinieren und diese dann dem Supportsuchenden weiter geben. Es könnte auch sein, dass der Bot dem Mitarbeiter überhaupt nicht helfen kann. Damit in diesen Fällen ein Supportmitarbeiter eingreifen kann, sollte der Bot in einer Chatgruppe laufen, von der ein Mitarbeiter des Support Teams ebenfalls Teil ist. Hierbei sollen die Fragen innerhalb eines Threads beantwortet werden, sodass nicht die jedes Mitglied der Gruppe eine Benachrichtigung bei jeder Nachricht des Chatbots erhält.

3.2.4 A4 Nutzung von Unternehmensdaten

Da die LLMs, die online auffindbar sind, keinen Zugriff auf spezifische Informationen der Techdivion haben, soll der Bot auf Unternehmensdaten zugreifen können. Dadurch kann der

Bot direkt spezifische Hilfe leisten und nicht nur allgemeine Hinweise geben, die im Internet zu finden sind, sondern hat auch Zugriff auf die Anleitungen aus dem internen Confluence.

3.2.5 A5 Erweiterbarkeit der Wissensdatenbank

Da sich die Unternehmensdaten mit der Zeit ändern und auch neue Datenquellen hinzukommen können, sollte der Bot in der Lage sein mit der Zeit auf neues Wissen zuzugreifen.

Das bedeutet, die Wissensdatenbank des Chatbots soll kontinuierlich erweitert werden können und bei Updates in den Datensystemen der Techdivsion regelmäßig aktualisiert werden können.

3.2.6 A6 Deutschsprachige Interaktion

Der Großteil der Techdivion ist deutschsprachig. Die meisten Teile von Datenquellen, wie beispielsweise Confluence sind ebenfalls auf Deutsch. Es gibt aber auch englischsprachige Mitarbeiter, daher wäre ein Bilingualer Bot der beste Fall, da aber die meisten Deutsch sind, wäre Deutsch die bessere Option, falls nur eine Sprache möglich ist.

3.2.7 A7 Logische & schlüssige Argumente

Die Argumente des Chatbots sollen logisch und schlüssig sein. Diese Anforderung hängt mit A4 zusammen. Der Chatbot soll die Fragen präzise ohne Halluzination und mit Hilfe der Unternehmensdaten beantworten.

3.2.8 A8 Speicherung von Anfragedaten

Dies ist eine optionale Anfrage des IT-Supports. Der Bot soll Anfragedaten speichern können, damit eingesehen werden kann, ob zu manchen Themen viele Fragen gestellt werden. So könnte, falls notwendig zu einem bestimmten Thema ein Workshop angeboten werden.

4 Konzeption

Bevor mit der Entwicklung begonnen werden kann, muss das Konzept erstellt werden. Zur Konzeption gehört in dieser Arbeit, die Auswahl der technischen Werkzeuge, die Programmiersprache, die Umgebung, die Modelle die benutzt werden sollen und die Architektur des Systems.

4.1 Auswahl der Modelle

Die erste Frage, die beantwortet werden muss, ist, welche Modelle benutzt werden sollen. Da innerhalb dieser Arbeit LLMs miteinander verglichen werden, werden fünf Modelle benötigt. Vier LLMs für den Chatbot selbst, der die Nachrichten generieren soll, die im Anschluss miteinander verglichen werden und ein Embedding Modell, welches für die Einbettungen der Vektordatenbank und der Anfragen zuständig sein wird.

4.1.1 Auswahl der Chat LLMs

Als erstes werden die Chat LLMs ausgwählt. Die Anforderung die hier beachtet werden muss, ist A1. Da der Bot lokal laufen soll, werden keine Modelle benutzt, die nur über eine externe API erreichbar sind, wie OpenAIs ChatGPT. Also ist die einzige Option ein Modell zu downloaden und mit diesem lokal zu interagieren.

In dieser Arbeit werden 4 verschiedene Modelle miteinander verglichen. Die Modelle können über die Seiten der Unternehmen, Huggingface (siehe 2.10 oder über Ollama (siehe 2.8) gedownloadet werden.

Innerhalb dieser Arbeit werden nur Modelle unter 14 Milliarden Parametern miteinander verglichen, da die Generationszeit kleinerer Modelle schneller ist und die meisten Informationen mit denen das Modell arbeiten soll innerhalb der Vektordatenbank gespeichert und über RAG dem Modell übergeben werden sollen.

Eine Plattform auf der LLMs miteinander verglichen werden ist das LMSYS Chatbot Arena Leaderboard. Hier werden die LLMs ähnlich wie bei Schach mit einem Elo Rating bewertet. Nutzer vergleichen die Ausgaben von zwei LLMs und bewerten diese. Von den Top 10 Modellen sind, bis auf Llama3 405B, alle proprietär und nur über eine API erreichbar. Unter den Ergebnissen außerhalb der Top 10 sind viele Versionen von Mistral, Llama3 und Gemma2. Manche von diesen sind finetuned und manche die Standardvarianten in verschiedenen Größen [Chi24].

Diese drei Modelle sind auch die Modelle, die auf Ollama die meisten Downloads haben. Daher wurden für diese Arbeit die drei Modelle

1. Mistral 7B von Mistral AI
2. Llama3 8B von Meta AI
3. Gemma 2 9B von Google

ausgewählt. Dazu kommt das zur Zeit der Auswahl neu veröffentlichte Phi3 Medium mit 14 Milliarden Parametern von Microsoft als letztes Modell dazu.

Diese Modelle wurden mit vielen Daten trainiert und dazu auch mit mehrsprachigen. Daher sind diese Modelle ohne weiteres Finetuning bereits in der Lage auf deutsch mit dem Nutzer zu interagieren und dadurch wird Anforderung A6 erfüllt.

4.1.2 Auswahl der Embedding Modelle

Nach der Auswahl der Chatmodelle geht es um die Auswahl des Einbettungsmodells. Es wäre hier möglich das extra Einbettungsmodell weg zu lassen und es mit den Chatmodellen einbetten zu lassen. Damit der Vergleich zwischen den verschiedenen Chatmodellen nicht durch die Einbettungen beeinflusst wird, wird für alle Chatmodelle das selbe Einbettungsmodell verwendet.

Ebenfalls wie bei den Chat Modellen werden hier Open Source Modelle benötigt, welche auf einem eigenen Server gehosted werden können.

Hierfür wird das Massive Text Embedding Leaderboard auf Huggingface betrachtet. Das MTEB Leaderboard sortiert Embedding Modelle nach ihrem durchschnittlichen Benchmark. Es gibt zur Zeit der Verfassung dieser Arbeit keine Möglichkeit nach deutschen Modellen zu filtern. Daher wird das aktuell generell beste mehrsprachige Modell in dieser Liste verwendet. Aktuell ist das das `multilingual-e5-large-instruct` Modell, von intfloat [Inta]. Dieses Modell baut auf dem Modell *lm-roberta-large"von FacebookAI auf. Das Model wurde finetuned indem es mit 1 Milliarde Datensätzen kontrastiv vortrainiert wurde und im zweiten Schritt wurde es mit den Datensätzen, die für das E5-Mistral Model in der Arbeit İmproving Text Embeddings with Large Language Models"verwendet wurden, trainiert [Wan24a].

4.2 Auswahl der Vektor Datenbank

Als nächstes kommt die Auswahl der Vektordatenbank. Für diesen Protoyp wird eine Vektordatenbank verwendet, die in einem Docker Container betrieben wird. Im Bezug auf die Auswahl der Vektordatenbank werden drei Punkte beachtet.

1. Open Source
2. Selbst hostbar
3. Integration in Langchain

Hierfür stehen die zwei Optionen zu Verfügung. Diese sind Chroma und Milvus. Beide Optionen sind Open Source und selbst hostbar [Chr, Hig]. Beide Optionen erfüllen auch die zweite Anforderung und haben eine Langchain Integration [Vec]. Durch die Integration innerhalb von Langchain, gibt es für beide Datenbanken innerhalb der Entwicklung des Chatbots eine ähnliche Bedienung. Beide Optionen erfüllen die Anforderungen die für die Vektordatenbank erstellt wurden. Aufgrund von persönlicher Erfahrung mit ChromaDB wird aus diesen Gründen ChromaDB für diesen Prototyp ausgewählt.

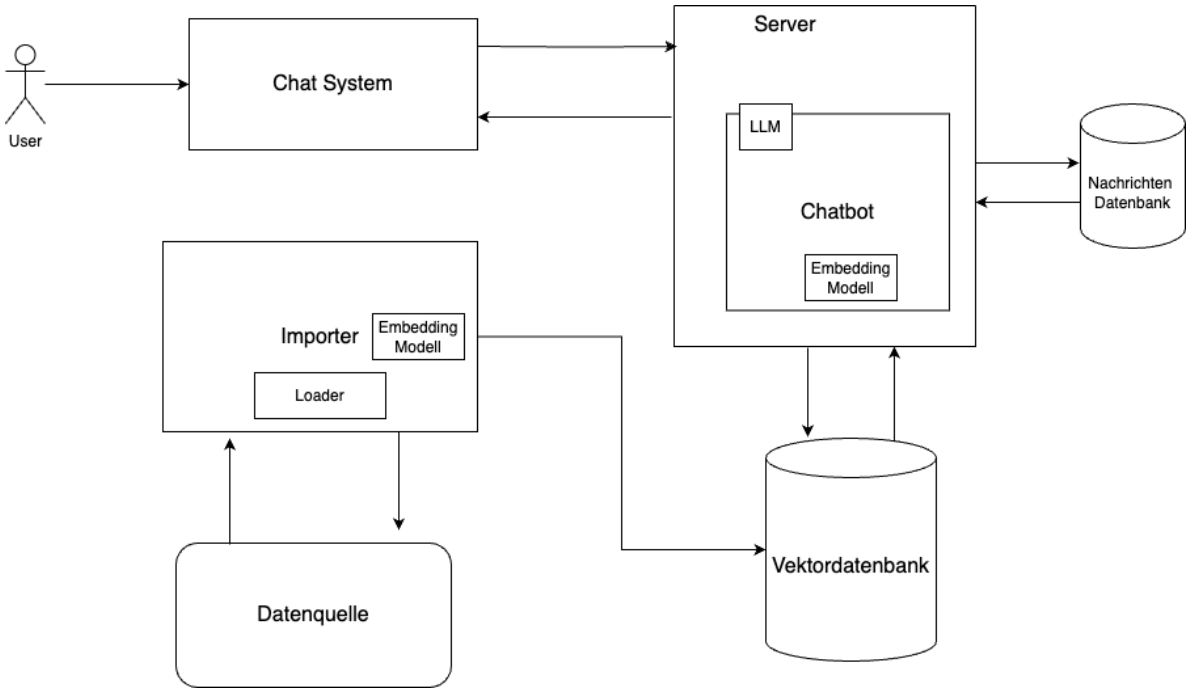


Abbildung 4.1 konzeptionelle Architektur

4.3 Architektur des Systems

Nachdem die Auswahl der Datenbank und der Modelle abgeschlossen ist, ist die nächste Frage, welche Architektur das gesamte System haben soll. In 4.1 ist der Aufbau des Systems dargestellt. Im Folgenden wird die Umsetzung der spezifischen Teile des Systems erörtert.

4.3.1 Verbindungen

Der Chatbot wird auf einem REST-Server laufen, welcher auf einem dedicated Server läuft, der genauer in 4.3.4 erklärt wird. Der Rest Server ist in der prototypischen Implementierung ein Python Server, der FastAPI nutzt.

Die Python Applikation ist mit einer Vektordatenbank verbunden, die auf dem selben Server in einem Docker Container in Betrieb genommen wird. Die Daten die auf den Docker Container geladen werden kommen von der Confluence Instanz der Techdivision. Um die Daten auf die Datenbank zu laden gibt es ein weiteres Python System, welches die Daten von der REST-API des Confluence Systems abfragt und diese in die Datenbank überträgt.

Die Anbindung der Vektordatenbank und der Aufbau des RAG-Systems wird in 4.3.3 erklärt.

Die Python Applikation wird anschließend mit der Google Chat API verbunden. Die Optionen über die die Verbindung hergestellt werden kann, werden in 4.3.5 besprochen.

4.3.2 Importer

Der Importer ist dafür zuständig, die Daten der jeweiligen Datenquellen in die Vektordatenbank zu laden. Hier ist es wichtig, dass die Datenquelle eine API hat von der die Einträge abgefragt werden können. Im Anschluss müssen die Einträge gereinigt werden und in ein

Format und in eine Länge gebracht werden, mit der das Framework, die LLM und die Vektor Datenbank umgehen können.

Die Daten werden im Anschluss durch ein Embedding Modell eingebettet und in der Vektordatenbank gespeichert.

4.3.3 Chatbot

Nach der groben Architektur des Systems kommt es zu der Architektur der einzelnen Teile. Begonnen wird hierbei mit dem Chatbot, also dem System, welches direkt auf dem Server laufen und das LLM hosten wird.

Framework

Bei der Auswahl der Konzeption für den Chatbot ist das erste Thema das zu verwendende Framework. Die drei Frameworks die zu Auswahl stehen, wurden in 2.9 vorgestellt. Diese sind Haystack, Langchain und LlamaIndex. Jedes der drei Frameworks ist nutzbar um einen nützlichen Chatbot mit RAG zu bauen. Bei allen drei gibt es die Möglichkeit mit RAG zu arbeiten und es ist ein Punkt auf den sich alle drei fokussieren [Whab, Llad, Intd].

Die Punkte die in Hinsicht auf das Framwork beachtet werden, sind die Größe des Projekts und ebenfalls die Menge an Dokumentation und Support. Da LLM Systeme sich aktuell schnell ändern und viele neue Erkentnisse in der Forschung gesammelt werden, verändern sich auch die KI Frameworks mit den LLM Systemen zusammen [Nav24]. Da das System möglichst langlebig sein soll und im besten Fall nur die einzelnen Bauteile, wie beispielsweise die LLM, mit der Zeit ausgetauscht werden sollten, ist es wichtig zu beachten, dass es ein Framework ist, welches möglichst lange bestehen bleibt und das Austauschen von einzelnen Teilen ohne großen Aufwand erlaubt. Dies hilft auch bei der Entwicklung bzw. Weiterentwicklung, da dadurch verschiedene Konfigurationen ausprobiert werden können ohne große Veränderungen durchführen zu müssen.

Die Community von Langchain ist die größte von den drei Framworks die in dieser Arbeit betrachtet werden, daher ist anzunehmen, dass der Support der Community und die Menge an Dokumentation, die gefunden werden kann hierbei am größten ist. Eine Github Suche nach Haystack gibt 1.7k Ergebnisse zurück. Eine Suche nach Langchain gibt 25k Ergebnisse zurück. Und eine Suche nach LlamaIndex gibt 1.3k Ergebnisse zurück. Eine Google Trends Analyse in diesem Bereich wäre vermutlich nicht sehr aussagekräftig, da der Begriff Haystack von verschiedenen Unternehmen benutzt wird und dazu noch ein normal gebrauchtes Wort ist, daher werden die Github Ergebnisse zur Orientierung genutzt. Hier werden zwar möglicherweise nicht alle Projekte aufgelistet, jedoch ist der Unterschied bereits hier merkbar. Langchain hat hier den Vorteil.

Das Austauschen von einzelnen Teilen des Systems ist mit allen Frameworks ähnlich aufwändig. Kleinere Änderungen können aufgrund der Chains in Langchain, der Pipelines in Llamaindex und der Pipelines in Haystack ähnlich leicht durchgeführt werden. Wenn eine größere Änderung durchgeführt werden soll, wie das Ändern von einem simplen Text-System zu einem multimodalen System, welches ebenfalls in anderen Formaten, wie Bild und Video arbeiten kann, dann würde es relevant aufwändige Änderungen in allen drei Frameworks erfordern. Daher wird hier angenommen, dass es bei allen drei Optionen den gleichen Aufwand kosten würde.

Hinzu kommt, dass Langchain und Llamaindex bereits Datenladesysteme für Confluence haben. Da Confluence das System ist, welches an die Vektordatenbank angebunden werden

soll, gibt das noch einen weiteren Vorteil. Hinzu kommt, wie an den Github Stars und den Github Ergebnissen erkennbar, dass Langchain das größte AI Framework ist und deshalb den meisten Support erhält. Aus diesen Gründen wird Langchain für diese Projekt ausgewählt.

Nachrichten Historie

Der Chatbot hat außerhalb der aktuellen Anfrage, der RAG Dokumente und des Prompts standardmäßig keine Erinnerungen oder Kontext. Falls in einer Nachricht auf den Kontext in einer vorherigen Nachricht eingegangen wird und dieser nicht explizit erwähnt wird, dann weiß der Chatbot nicht worum es geht. Dies führt dazu, dass keine Folgefragen gestellt werden können, ohne dass der vorherige Kontext erneut erklärt wird.

Um dieses Problem zu lösen muss eine Nachrichten Historie mit in das System eingebaut werden. Das Framework Langchain, welches in 4.3.3 festgelegt wurde, gibt einem die Möglichkeit eine Nachrichtenhistorie einzubauen.

Den abgespeicherten Chats muss eine Session ID gegeben werden, damit die Chats abgerufen werden können.

Chats innerhalb des Spaces werden innerhalb von Threads geführt. Google gibt den Threads bereits eine ID. Da der Bot dieser Arbeit nur innerhalb von Spaces arbeiten soll, wird diese ID verwendet um einen Python Hash Wert zu erzeugen. Dieser wird als Session ID verwendet. Diese Herangehensweise funktioniert nur innerhalb der Spaces, da die API über HTTPS-Server nicht in der Lage ist, auf Threads innerhalb von Einzelchats zu antworten.

Dieser Ansatz speichert die Nachrichten und erfüllt dadurch Anforderung 3.2.8.

RAG

Der Chatbot muss, um die durch den Importer (siehe 4.3.2) in die Vektordatenbank geladenen Daten sinnvoll nutzen zu können mit RAG ergänzt werden. Hierfür wird das in 4.1.2 ausgewählte Embedding Modell verwendet und es wird eine Verbindung zur Vektordatenbank hergestellt. Über diese Verbindung werden die relevanten Dokumente von der Vektordatenbank angefragt und werden im Anschluss der LLM mitgeschickt.

4.3.4 Server

Bei der Auswahl des Systems muss beachtet werden, dass die Techdivision den Server für weitere AI Projekte verwenden will. Daher wird ein Server benötigt, der den Chatbot und die Datenbank, ohne ausgelastet zu sein, betreiben kann.

Das größte Modell das innerhalb dieser Arbeit verwendet wird, ist Phi-3 Medium von Microsoft, welches 14 Milliarden Parameter hat. Die Menge an VRAM, die ein Modell dieser Größe in voller Präzision benötigt ist 56GB VRAM. Die Standard Modelle, die von Ollama heruntergeladen werden, sind 4 Bit quantisiert, wodurch die Anforderung auf 7GB VRAM sinkt. Die Uvicorn Worker, welche verwendet werden um den FastAPI Server als Service zu betreiben brauchen jeweils 2GB VRAM und RAM. Da der Server nicht alleine durch den Bot ausgelastet sein soll, wird nach einem Server mit einer VRAM Größe von mindestens 16GB gesucht. Dazu werden 64GB RAM benötigt. Hierfür stehen zwei Optionen zur Verfügung.

1. Eine Compute Engine Instanz in der Google Cloud
2. Ein dedicated Hetzner Server

Der Hetzner Server der zur Auswahl steht ist der aktuell einzige dedicated GPU Server, der bei Hetzner angeboten wird [Het]. Er hat einen Intel Core i5-13500 als CPU mit 6 Performance Cores und 8 Efficient Cores. 64 GB DDR4 RAM, 2x 1.92 TB Gen3 NVMe SSDs und eine Nvidia RTX 4000 SFF Ada Generation mit 20GB GDDR6 ECC. Die Kosten für diesen Server wären maximal, wenn der Server 730 Stunden im Monat läuft, 218.96€ + 94.01€ einmalige Setupgebühr.

Diese Hardware ist ausreichend für die Anforderungen des Systems und ermöglicht ihn für mehr Applikationen als diesen Chatbot zu nutzen.

Wenn ein ähnlicher Server in der Compute Engine von Google zusammengestellt wird, dann gibt es aktuell die Möglichkeit GPUs in den Maschinen Kategorien Allzweck, Rechen optimiert und Beschleunigungsoptimiert hinzuzufügen. Für die Berechnung der Kosten wird erneut angenommen, dass der Server 730 Stunden im Monat läuft. Wenn ein N1 Allzweck Server mit 64GB RAM, 10 vCPUs, welche mindestens benötigt werden um 64 GB RAM auswählen zu können, und eine Nvidia T4 GPU mit 16GB VRAM ausgewählt wird, kostet diese Compute Engine Instanz 650€ pro Monat. Der Hetzner Server, ist bei voller Nutzung ca 430€ billiger im Monat und hat dazu 4GB mehr VRAM, daher wird der Hetzner Server ausgewählt.

4.3.5 Anbindung Google Chat

Die Anbindung an den Google Chat erfolgt über die Google Chat API. Hierbei gibt Google den Nutzern vier Optionen dafür.

Die vier Optionen sind:

1. App URL mittels https Server mit authentifiziertem SSL Zertifikat
2. Apps Script Projekt
3. Cloud Pub/Sub
4. Dialogflow

Dialogflow ist keine Option, da hier nicht die Option besteht eigene Open Source LLMs zu hosten und stattdessen die von Google angebotenen LLMs genutzt werden müssten, wodurch Anforderung A1 verletzt werden würde.

Apps Script ist auch keine Option, da bei Apps Scripts nur ein Skript ausgeführt wird, und keine vollwertige Umgebung zur Verfügung steht. Dies ist zu limitierend für diese Arbeit.

Da bereits ein HTTPS REST Server geplant ist, wird hier die App URL verwendet, wodurch direkt mit der API interagiert werden kann und die API Calls, bei ankommenden Nachrichten, direkt an den Server gesendet werden.

Pub/Sub wäre eine Alternative, falls eine Firewall im Weg stehen würde, was hier aber nicht der Fall ist, daher ist dieser Zwischenschritt nicht notwendig.

Für Google Chat gibt es die Möglichkeit Nachrichten synchron und asynchron zu versenden. Für eine synchrone Nachricht muss die Antwort innerhalb von dreißig Sekunden an die API gesendet werden, nachdem eine Nachricht erhalten wurde. Für den Nutzen innerhalb dieser Arbeit wäre dies genug, aber über synchrone Nachrichte, gibt es keine Möglichkeit auf Threads zu antworten, stattdessen werden einzelne Nachrichten versendet. Da die Threads benötigt werden, um einen sinnvollen Kontext für den Chatbot über die Nachrichtenhistorie zu erstellen, wird hier der asynchrone Ansatz gewählt.

Die Anbindung an Google Chat erfüllt dadurch die Anforderungen A2 und A3.

4.3.6 Erfüllung der Anforderungen

Durch Nutzung einer lokalen LLM, die deutschsprachige Interaktion unterstützt, werden die Anforderungen A1 und A6 erfüllt. Die Google Chat Anbindung ermöglicht die Umsetzung von A2 und A3. Die Einbindung einer Vektordatenbank mit RAG und das Nutzen einer Importers für das Importieren der Daten erfüllen A4 und A5. Das Einbauen der Nachrichten Historie addressiert A8. Die Anforderung A7 wird nicht direkt von einer Komponente abgedeckt und soll in der abschließenden Evaluation mithilfe des Fragebogens beurteilt werden.

5 Entwicklung

Nach dem Abschluss der Konzeption kann zur Entwicklung über gegangen werden. Die Rahmenbedingungen hierbei sind, dass der Hetzner Server, welcher in 4.3.4 beschrieben wurde verwendet wird und auf diesem Ubuntu 22 installiert ist.

5.1 Vorbereitung

Bevor mit der Entwicklung begonnen werden kann, müssen ein paar Vorbereitungen getroffen werden.

5.1.1 Modelle

Der erste Schritt ist es die Modelle vorzubereiten.

Um die Modelle vorbereiten zu können, wird als erstes Ollama installiert. Dies kann über das Install Skript, welches auf Github zu finden [Oll24] ist getan werden. Dazu werden noch die Nvidia Cuda Driver installiert, damit Ollama diese nutzen kann. Nun können die Modelle vorbereitet werden.

Hierfür werden die Modelle, die in 4.1 vorgestellt wurden über Ollama gedownloadet. Die Modelle werden über `Ollama pull model` geladen, wobei `model` der Name des Modells ist, welches heruntergeladen werden soll. Hierbei kann mittels `model:latest` noch die Version, die benötigt wird, angeben werden, wobei `latest`, die Version des Modells ist. Darüber kann ebenfalls die Größe oder spezielle Version des Modells angefragt werden. Die verschiedenen Modelle haben verschiedene Optionen. Auf Ollama.com können die jeweiligen Modelle und die verschiedenen Versionen, eingesehen werden.

5.1.2 Google Cloud

Hier geht es um die Vorbereitung des Google Cloud Projekts für die Verbindung mit der Google Chat API.

Der erste Schritt hierbei ist es, ein Google Cloud Projekt zu erstellen. Wenn dies erledigt ist, muss die Google Chat API innerhalb des Projekts aktiviert werden.

Dazu wird ein Google Service Account benötigt, der auf die Google Chat API zugreifen kann. Die Konfiguration der API wird stattfinden, nachdem der Server konfiguriert wurde und das System funktionsfähig ist.

5.1.3 Datenbank

Hierfür wird auf dem Server Docker installiert. Dazu wird `chromadb/chroma` über `docker pull` heruntergeladen und dann mittels `docker run` gestartet.

5.2 System

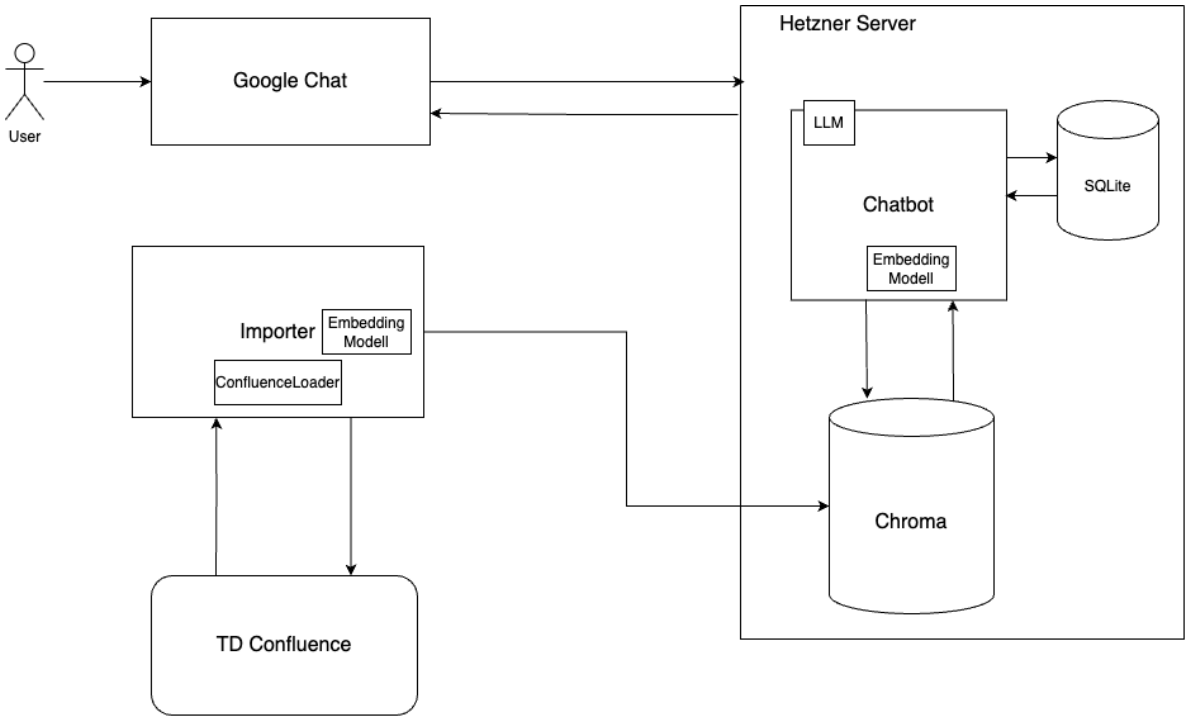


Abbildung 5.1 Konkrete Architektur

Wie in der Konzeption beschrieben, ist das System auf dem der Chatbot erstellt wird ein Hetzner Server auf dem ebenfalls die Datenbank liegt. Nachdem die Vorbereitungen des Servers abgeschlossen sind kann mit der eigentlichen Entwicklung begonnen werden.

In 5.1 ist die Architektur mit den genutzten Komponenten dargestellt.

Für die Entwicklung wird innerhalb dieser Arbeit erst Pseudo-Code gegeben und im Text wird dann die genaue Entwicklung mit den benutzten Frameworks beschrieben.

5.2.1 Importer

Algorithm 1 Importer main

1: **Methode** main():
2: Initialisiere Konfiguration, Dokumenten-Loader, Dokumenten-Splitter, Embedding Modell und Vektor-Datenbank
3: Lade Dokumente aus Confluence
4: Teile Dokumente auf
5: Speichere aufgeteilte Dokumente in Vektor-Datenbank

Algorithm 1 zeigt den generellen Ablauf des Importers. Für den Importer wird der Langchain `ConfluenceLoader` verwendet. Dieser wird über die DocumentLoader Klasse in Algorithm 2 gesetzt. Hierfür wird, nachdem der Docker Container, welcher die ChromaDB Datenbank beinhaltet gestartet wurde, eine Verbindung zu diesem hergestellt. Dabei wird dem Chroma Client innerhalb des Python Programms der `host` und der `port` des Docker Containers in Algorithm 5 übergeben.

Algorithm 2 Dokumenten-Loader erstellen und Dokumente laden

```
1: Klasse DocumentLoader:
2: Methode __init__(config):
3:     Erstelle Document Loader
4: Methode load_documents(space_key, limit):
5:     Lade Dokumente aus Confluence
6:     Rückgabe: Geladene Dokumente
```

Algorithm 3 Dokumenten-Splitter erstellen und Dokumente aufteilen

```
1: Klasse DocumentSplitter:
2: Methode __init__():
3:     Definiere Header zum Aufteilen
4:     Erstelle Splitter
5: Methode split_documents(docs):
6:     Teile Dokumente auf:
7:         Initialisiere leere Liste für aufgeteilte Dokumente
8:         Für jedes Dokument in docs:
9:             Teile Dokument mit Splitter
10:            Füge Metadaten hinzu
11:            Füge aufgeteilte Dokumente zur Liste hinzu
12:    Rückgabe: Aufgeteilte Dokumente
```

Algorithm 4 Embedding-Modell erstellen

```
1: Klasse EmbeddingModel:
2: Methode __init__(config):
3:     Lade Embedding Modell mit Modellname und Konfigurationsparametern
```

Algorithm 5 Vektor-Datenbank erstellen und Dokumente speichern

```
1: Klasse VectorDatabase:
2: Methode __init__(config):
3:     Erstelle Vektor-Datenbank Client
4: Methode save_documents(docs, embeddings, collection_name):
5:     Speichere Dokumente in Vektor-Datenbank:
6:         Erstelle Vektor-Datenbank Instanz mit Dokumenten und Embeddings
7:         Persistiere Vektor-Datenbank
8:     Rückgabe: Vektor-Datenbank Instanz
```

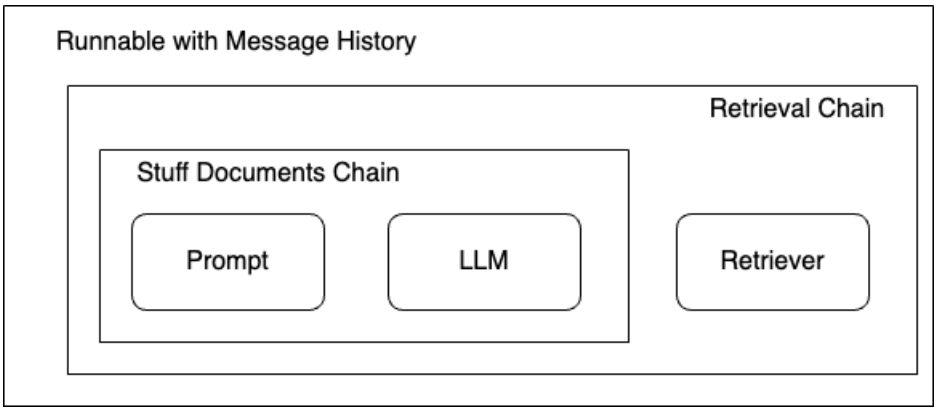


Abbildung 5.2 Aufbau der Chain

Danach wird das Embedding Modell in Algorithm 4 über die `HuggingFaceEmbeddings` von Langchain geladen. Der Wert `intfloat/multilingual-e5-large-instruct` wird dafür übergeben. Dazu gibt es die Option Keyword Arguments mittels `encode_kwargs` und `model_kwargs` mitzugeben. Hier wird das `device_kwargs device` mit dem Wert `cuda` gesetzt. Dies sorgt dafür, dass die GPU für die Berechnungen verwendet wird, was den Prozess beschleunigt.

Nun können die Daten aus Confluence mittels `ConfluenceLoader` geladen werden. Hier werden `username`, `api_key` und die `url` der Confluence Instanz benötigt. Nach dem Erstellen des Loaders können die Daten geladen werden, hierfür wird die `load` Funktion, welche einen `space_key` braucht, für den Confluence Space der geladen werden soll, verwendet.

Die geladenen Dokumente, werden wie in Algorithm 3 dargestellt über Splitter aufgeteilt. Hierfür werden die Dokumente mittels `MarkdownHeaderTextSplitter` ein mal aufgeteilt und mittels `RecursiveCharacterTextSplitter` weiter aufgeteilt. Den aufgeteilten Dokumenten werden die Metadaten hinzugefügt, damit sie zugeordnet werden können.

Anschließend wird eine Langchain Chroma Instanz erstellt, die auf den vorher bereits erstellten Client zugreift, dann werden die Dateien mittels `from_documents` innerhalb einer Chroma Collection, die beim Erstellen über die Chroma Instanz mittels `collection_name` festgelegt wird, gespeichert.

5.2.2 Chatbot

Die Entwicklung des Chatbots wird in die einzelnen Teile der Langchain Chain, die in 5.2 dargestellt ist, aufgeteilt. Die Chain besteht aus verschiedenen Chains, in denene Langchain Elemente aneinander gekettet wurden. Das Prompt und die LLM werden mittels `create_stuff_documents_chain` miteinander verbunden. Diese Chain wird mittels `create_retrieval_chain` mit einem Retriever verbunden. Als letztes wird diese Chain in eine `RunnableWithMessageHistory` eingebaut. Im Folgenden werden die einzelnen Teile erklärt.

Retriever

Der erste Teil der Chain ist der `Retriever`. Der Retriever ist der Teil der Chain, der auf die Vektordatenbank zugreift und die passenden Dokumente von dieser an die LLM

Algorithm 6 Retriever Initialisierung

```
1: Class RetrieverManager:
2: Method __init__(model_name, model_kwargs, encode_kwargs, client_settings):
3:     Initialisiere Vektorddatenbank Client
4:     Initialisiere Embeddings
5:     Erstelle Vektordatenbank Instanz mit Embeddings und Client
6:     Erstelle Retriever mit Vektordatenbank Instanz
```

übergibt.

Der Retriever wird über den RetrieverManager in Algorithm 6 konfiguriert. Hierfür wird, wie beim Importer in 5.2.1 ein Chroma Client erstellt, der mit dem Docker Container der Chroma Instanz verbunden wird, dazu wird ebenfalls, wie beim Importer, das Embedding Modell über `HuggingFaceEmbeddings` geladen und in der Chroma Instanz als `embedding_function` angegeben. Dieser wird dann mit der Funktion `as_retriever()` als Retriever geladen. Das wichtige hierbei ist es den selben `collection_name` wie beim Importer anzugeben.

Prompt

Algorithm 7 Initialisierung des Chatbot Prompts

```
1: Class ChatbotPrompt:
2: Method __init__():
3:     Definiere Prompt
```

Das Prompt wird über Algorithm 7 mittels `ChatPromptTemplate.from_messages` erstellt. Das Prompt, welches benutzt wird, ist Folgendes:

Dein Name ist Arthur. Du bist ein erfahrenes Mitglied des IT-Support-Teams der Firma TechDivision. Deine Aufgabe ist es, Mitarbeitern bei ihren IT-Problemen zu helfen. Beachte folgende Richtlinien:

1. Nutze ausschließlich die Informationen aus dem bereitgestellten Kontext und deinem vorhandenen Wissen über allgemeine IT-Konzepte, um Fragen zu beantworten.
2. Wenn die Antwort nicht im Kontext oder in deinem allgemeinen IT-Wissen enthalten ist, sage ehrlich, dass du die Information nicht hast oder die Frage nicht beantworten kannst.
3. Erfinde oder halluziniere keine Informationen.
4. Beziehe dich auf frühere Teile der Konversation, wenn sie relevant sind.
5. Bleibe stets höflich, professionell und hilfsbereit.
6. Wenn nötig, frage nach weiteren Details, um das Problem besser zu verstehen.
7. Gib schrittweise Anleitungen, wenn du eine Lösung vorschlägst.
8. Verweise den Mitarbeiter an andere Abteilungen oder Experten, wenn das Problem außerhalb deines Zuständigkeitsbereichs liegt.

9. Antworte auf Deutsch.

Kontext: {`context`} Bisheriger Chatverlauf: {`chat_history`} Beantworte die folgende Frage des Mitarbeiters basierend auf diesen Informationen.

Wobei `context`, die über die Chain übergebenen RAG Dokumente sind und `chat\_history` die über die Chain übergebene Chat Historie ist. Die Richtlinien, sollen der LLM eine Richtung geben und dazu falsche Informationen und Halluzinationen vermeiden.

Nachrichten Historie

Algorithm 8 Nachrichten Historie

```
1: Method get_session_history(session_id):
2:     Rückgabe einer Session mit Session Id
```

Die Nachrichten Historie wird mittels `RunnableWithMessageHistory` implementiert. Hierfür wird noch eine `get_session_history` Funktion, wie in Algorithm 8 dargestellt, benötigt. Diese ruft über eine SQLite Datenbank mittels der `session_id` und der `SQLChatMessageHistory` Klasse die aktuellen Session auf.

LLM

Algorithm 9 LLM-Manager

```
1: Class LLMManager:
2: Method __init__(model_name):
3:     Initialisiere Sprachmodell
```

Dieser Schritt kann erst funktionieren, wenn die in 5.1.1 beschriebenen Vorbereitungen abgeschlossen sind. Das LLM wird dafür mit der `Ollama` Community LLM in der LLM-Manager Klasse (siehe Algorithm 9) geladen. Wobei der Klasse der Name des Modells aus Ollama übergeben wird.

5.2.3 Google Cloud API

Algorithm 10 Google Chat Client

```
1: Class GoogleChatClient:
2: Method __init__(credentials_file, scopes):
3:     Initialisiere Google Chat API-Anmeldedaten aus Datei und richte Chat-Client ein
4: Method send_message(space_name, thread_name, user_name, message):
5:     Sende Nachricht an den angegebenen Space und Thread in Google Chat
```

In Algorithm 10 ist der Pseudo Code für den Google Chat Client dargestellt. Um async Nachrichten an die Google Chat API zu senden, wird eine Credential Datei, von einem Service Account, der mit der Google Chat API verbunden ist, benötigt. Im Code wird der Scope `https://www.googleapis.com/auth/chat.bot` spezifiziert und aus dem Scope

und dem Service account wird mit der `build` Funktion der client erstellt. Die Nachricht wird mit der Funktion `Client.spaces().message().create().execute()` erstellt und abgesendet. Die Funktion bekommt die notwendigen Informationen für die Nachrichten übergeben. Aus diesen wird die Nachricht aufgebaut und mit der `messageReplyOption REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD` abgesendet. Diese sorgt dafür, dass, falls ein Thread, der existiert, angegeben ist, im jeweiligen Thread geantwortet wird und sonst wird außerhalb des Threads geantwortet. Das finale Objekt, welches an die API geschickt wird ist in A.1 dargestellt.

5.2.4 Post Endpunkt

Algorithm 11 POST Endpunkt

```
1: Definiere POST-Endpunkt /:
2:     Empfange und parse JSON-Anfrage
3:     Extrahiere Nachrichtentext, Benutzerinformationen, Thread-Name und E-Mail-Host
4:     Überprüfe, ob der E-Mail-Host gültig ist
5:         Wenn nicht gültig:
6:             Rückgabe einer Fehlermeldung
7:     Generiere Sitzungs-ID basierend auf dem Thread
8:     Initialisiere Retriever mit angegebenen Einstellungen
9:     Initialisiere ChatbotPrompt zur Abrufung der Promptvorlage
10:    Initialisiere LLMManager mit Modell
11:    Erstelle eine Chain aus Modell und Prompt
12:    Erstelle eine weitere Chain aus dem Retriever und der vorherigen Chain
13:    Initialisiere Session History Chain
14:    Führe Kette aus, um Antwort auf Benutzereingabe zu erhalten
15:    Initialisiere GoogleChatClient mit Anmeldedaten
16:    Verwende GoogleChatClient, um Antwortnachricht in Google Chat mit Benutzername
       und Antwort zu senden
17:    Rückgabe einer leeren Antwort
```

Nachrichtenverarbeitung

Für die Verbindung an die Google Cloud API wird ein POST Endpunkt erstellt. Der Pseudo Code des POST Endpunkts ist in Algorithm 11 dargestellt. Dieser Endpunkt nimmt die Nachrichten, die in Google Chat an den Chatbot gesendet werden an. Die JSON Nachricht, die von der API abgeschickt wird beinhaltet alle notwendigen Informationen. Für den Chatbot sind die ID des Nutzers, der die Nachricht abschickt, der Thread in dem die Nachricht versendet wurde und die Nachricht, die abgesendet wurde, wichtig. Dazu wird noch der Email Host, von dem Nutzer von dem die Nachricht kommt, überprüft. In A.0 ist das Format der Nachricht, welche der Chatbot erhält, dargestellt.

Der Thread ist unter `message.thread.name` zu finden. Die Thread ID, welche der letzte Teil davon ist, wird verwendet um die Session ID herzustellen und sie wird in der Antwort als `thread.name` an die API geschickt.

Chatbot Chain

Nachdem die Session ID erstellt wurde, werden aus dem Modell des LLMManagers und dem Prompt des ChatbotPrompts mit der Funktion `create_stuff_document_chain()` eine Stuff Document Chain erstellt. Mit dieser und dem Retriever aus dem RetrieverManager wird über die Funktion `create_retrieval_chain()` eine Retrieval Chain erstellt und daraus wird final die `RunnableWithMessageHistory` erstellt.

```
RunnableWithMessageHistory(
    retrieval_chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="chat_history",
    output_messages_key="answer",
)
```

Abbildung 5.3 RunnableWithMessageHistory

Der `RunnableWithMessageHistory` muss ein Runnable Objekt, welches als Basis dient, übergeben werden. Das Schema für `RunnableWithMessageHistory` ist in 5.3 dargestellt. Hier ist das Basis Runnable Objekt, die Retrieval Chain. Dazu wird der Runnable die `get_session_history` Funktion aus Algorithm 8 übergeben. Da das Basis Runnable ein dict als Input benötigt, muss ein `input_messages_key` angegeben werden, was laut dem Prompt, welches verwendet wird der Key `input` ist. Dazu wird der `output_messages_key` `answer` übergeben und der `history_messages_key` der Wert `chat_history`. Der `chat_history` Wert wird aus dem Prompt abgeleitet.

Dieses `RunnableWithMessageHistory` Runnable kann mit `runnable.invoke()` gestartet werden. Der Funktion werden zwei dict Objekte mitgegeben. Das dict mit dem `input` Key, der die Eingabe des Nutzers ist und dazu noch ein weiteres dict mit einem `configurable` Key, dessen Wert ein dict ist, mit einem `session_id` key und der jeweiligen `session_id`.

Absenden der Nachricht

Die ID des Nutzers, die unter `message.sender.name` zu finden ist, wird benutzt um den Nutzer in der Nachricht anzusprechen. Die Nachricht, die abgesendet wird beginnt hierfür mit `<message.sender.name>` und dadurch kann über die API ein Nutzer angesprochen werden. Dies funktioniert nur über die ID. Wenn stattdessen der DisplayName aus `message.sender.displayName` verwendet wird, dann wird der Nutzer nicht in der Nachricht verlinkt.

Ein GoogleChatClient wird initialisiert, die notwendigen Infos werden an diesen übergeben und die Nachricht wird abgesendet. Das wichtige hierbei ist, dass, wenn eine asynchrone Nachricht abgeschickt wird, am Ende eine leere Antwort als Rückgabewert angegeben wird. Wenn diese leere Antwort nicht zurück gegeben wird, kann es aktuell passieren, dass die API eine Inaktivität des Bots meldet und die asynchrone Nachricht erst danach ankommt.

5.2.5 Server Konfiguration

Damit die Google Chat API den Server akzeptiert, wird ein SSL Zertifikat benötigt. Das SSL Zertifikat wurde über certbot generiert und es wurde Nginx verwendet um das SSL Zertifikat zu laden und als Reverse-Proxy für den FastAPI Server zu nutzen. Der FastAPI Server wird als Service gestartet und für die Demo wurden 4 Unicorn Worker, für den FastAPI Server benutzt.

5.2.6 Google Chat API Konfiguration

Nachdem der Chatbot fertig gestellt wurde und der Server ein SSL Zertifikat und eine URL besitzt, kann die Google Chat API Konfiguration fertig gestellt werden.

Der API muss ein Anwendungsname gegeben werden. Über diesen muss die API im Chat angesprochen werden. Dazu wird eine URL zu einem Bild, welches das Profilbild im Chat sein wird, benötigt. Dazu muss eine Beschreibung des Projekts angegeben werden.

Die interaktiven Features müssen aktiviert sein und die Option Gruppenbereichen und Unterhaltungen beitreten muss ausgewählt sein.

Als Verbindungseinstellung wird Anwendungs-URL ausgewählt und es wird die URL zum Server eingetragen.

Da dies nur ein Prototyp ist, wird dieser noch nicht auf dem Google Workspace Marketplace veröffentlicht. Da Nutzer die App bewerten werden brauchen diese Zugriff. Diese können im Abschnitt `Sichtbarkeit` manuell eingetragen werden. Damit ist die Entwicklung des Chatbots und die Konfiguration der API abgeschlossen.

5.2.7 Chatbotinteraktion

Der Chatbot kann nun über Google Chat wie jeder andere Teilnehmer des Workspaces angeschrieben werden. Wenn der Chatbot einem Space hinzugefügt werden soll, dann muss dieser als App hinzugefügt werden und nicht als Nutzer.

In A.2 ist eine Interaktion innerhalb eines Threads dargestellt, in der die Erinnerungen des Bots angefragt werden und es ist erkennbar, dass der Bot sich an die vorherige Frage erinnern kann.

6 Vergleich

6.1 Evaluierung

Der abschließende Teil der Arbeit fokussiert sich auf den Vergleich des Systems mit verschiedenen LLMs. Hierfür wurde das System mit vier verschiedenen LLMs implementiert und wird zunächst mithilfe des RAGAS-Frameworks evaluiert. Im Anschluss an diese Analyse werden die zwei bestabschneidensten Modelle einer weiteren Bewertung durch Mitarbeitern der Techdivision unterzogen. Während im Konzept bereits die meisten Anforderungen aus 3.2 erfüllt wurden, werden in diesem Teil A4 und A7 erneut aufgegriffen.

Modelle

Die vier LLM Modelle, die bewertet werden sind Metas Llama3, Mistrals Mistral, Googles Gemma 2 und Microsofts Phi-3.

Der größte Unterschied zwischen diesen Modellen ist die Parametergröße. Mistral hat hierbei 7 Milliarden Parameter, Llama3 8 Milliarden, Gemma 2 9 Milliarden und Phi-3 ist das größte Modell mit 14 Milliarden Parametern.

Alle Modelle basieren auf der Decoder Only Transformer Architektur, unterscheiden sich jedoch in ihren Attention Mechanismen. Mistral verwendet sliding window attention (siehe 2.3.1), während Gemma2 zwischen sliding window und global attention variiert. Phi-3 hingegen nutzt blocksparse attention (siehe 2.3.1). Alle Modelle haben gemeinsam, dass Grouped-query Attention (siehe 2.3.1) verwendet wird. Die Modelle unterscheiden sich ebenfalls in der Tokenmenge, die für das Training verwendet wurde. Gemma 2 wurde mit 8 Billionen Token trainiert, Llama3 mit 15 Billionen und Phi-3 mit 4.8 Billionen [Mis, Intc, Abd24, Gem24]. Die Menge von Token für Mistral ist nicht bekannt.

6.2 RAGAS Vergleich

Ragas ist das Framework, welches zum Bewerten und Vergleichen der LLMs innerhalb dieser Arbeit verwendet wird. Das Framework dient dazu RAG Pipelines zu bewerten. Im Rahmen davon lassen sich Testsets generieren, die Fragen, Antworten und die dazu passenden RAG Dokumente beinhalten.

Über diese Testsets lassen sich die Pipelines über verschiedene Metriken vergleichen. Die Metriken die zur Verfügung stehen und die im finalen Vergleich relevant sind, werden in 6.2.2 genauer beschrieben.

Die erste Phase des Evaluierungsprozesses findet über dieses Framework statt.

6.2.1 Testset

Der erste Schritt für den Vergleich ist es Testsets herzustellen anhand derer Ragas den Benchmark Test durchführen kann.

Die Variante die hierfür verwendet wird ist es mit `TestsetGenerator.from_langchain()` einen TestsetGenerator herzustellen und mit diesem das Testset erstellen zu lassen.

Als Vorbereitung hierfür werden Daten benötigt. Die Daten die als Dokumente geladen werden haben alle ein Metadata Dictionary. Damit der Generator erkennen kann, welche Daten zusammengehören brauchen die Dokumente in diesem Metadata Dictionary ein Key Value Pair mit dem Schlüssel `filename`. Der Wert des `filename` Schlüssels wird verwendet um bei der Generation der Testdaten zusammengehörige Dokumententeile zusammenzufügen.

Nach dem Vorbereiten der Daten kann damit begonnen werden den Generator zu erstellen. Für diesen Generator werden drei Parameter verwendet. Die `generator_llm`, `critic_llm` und `embeddings`. Als Embedding Modell wird, wie für den Chatbot und den Importer, ebenfalls das Modell `intfloat/multilingual-e5-large-instruct` genutzt. Als Critic und als Generator LLM wird das 70 Milliarden Parameter Model Llama3 von Meta verwendet. Die Nutzung von Hugginface Modellen mit Ragas funktioniert aktuell nicht, daher wird das Llama3 Modell, welches direkt über Ollama gedownloadet werden kann, verwendet. Nach dem Erstellen des Generators und dem Vor und Aufbereiten der Daten kann das Testset über die Funktion `generate_with_langchain_docs()` generiert werden. Für die Generierung werden drei Parameter benötigt. Als erstes die `documents` aus denen die Testsets generiert werden sollen. Hier werden Dokumente verwendet die über den `ConfluenceLoader` von Langchain geladen werden. Dies wird genau wie beim Importer (siehe 5.2.1) umgesetzt. Dazu kann bestimmt werden, wie viele Elemente im Set enthalten sein sollen. Diese `test_size` bestimmt wie viele Fragen vom `TestsetGenerator` generiert werden. Dazu kann die Verteilung der Fragen bestimmt werden. Die Fragen die durch den Generator erstellt werden können in diesem Zusammenhang drei verschiedene Typen haben. Diese drei Typen sind: `simple`, `reasoning` und `multi_context`. Diese können, der `generate` Funktion, mit der Verteilung die verwendet werden soll, als dict übergeben werden. Hier werden diese Werte verwendet: `simple: 0.5`, `reasoning: 0.2` und `multi_context: 0.3`.

Das Testset welches hierbei generiert wird kann mittels `to_pandas()` als pandas Dataframe ausgeben werden. Wenn dieses Dataframe betrachtet wird, werden 4 relevante Spalten angezeigt.

Diese sind:

`question`: Die Frage die dem Chatbot für das Benchmarking gestellt wird.

`context`: Der Context aus dem die Frage erstellt wurde.

`ground_truth`: Die Antwort auf die Frage.

`question_type`: Der Question Typ, dessen Verhältnis in der `generate` Funktion bestimmt wurde.

Mit diesem Testset kann die Chain bewertet werden.

6.2.2 Evaluation

Nachdem das Testset generiert wurde kann mit der Bewertung begonnen werden. Für die Bewertung gibt es acht verschiedene Metriken, die zur Auswahl stehen. Hiermit können die verschiedenen Teile der Rag-Pipeline bewertet werden. Einige der Metriken bewerten nur den RAG Aspekt der Chain, wobei nur die abgerufenen Dokumente bewertet werden. Da in diesem Aspekt innerhalb dieses Vergleichs keine Änderungen stattfinden, sondern nur die LLMs die zur Generation der Antworten genutzt werden, geändert werden, werden nicht alle Metriken betrachtet. Im Folgenden werden die Metriken die zu zur Bewertung

verwendet werden vorgestellt. Standardmäßig verwendet die `evaluate` Funktion die OpenAI API, dies kann überschrieben werden indem der Funktion ein `llm` und ein `embedding` übergeben wird. Die Llm und das Embedding Model können in `LangchainLLMWrapper` und in `LangchainEmbeddingsWrapper` verpackt werden, falls kein Langchain LLM oder Embedding verwendet wird. Wenn die LLM und das Embedding von Langchain sind, dann können diese direkt übergeben werden. Diese Funktion ist aktuell aber nicht funktionsfähig, daher wird hierfür die OpenAI API verwendet.

Faithfulness

"Faithfulness" zu Deutsch "Genauigkeit" ist die Metrik, die die generierte Antwort mit dem abgerufenen Kontext vergleicht. Dieser Wert ist eine Gleitkommazahl zwischen 0 und 1. Hierbei werden erst die Behauptungen aus der Antwort heraus gesucht und diese werden dann mit dem abgerufenen Kontext verglichen und es wird überprüft, ob die Behauptung aus dem Kontext gezogen werden kann [Ragd].

Die Formel die hierfür verwendet wird ist wie folgt:

$$\text{Faithfulness score} = \frac{|\text{Anzahl der Behauptung die aus dem Kontext geschlossen werden können}|}{|\text{Gesamtzahl der Behauptungen}|}$$

Answer relevance

"Answer relevance" zu Deutsch "Relevanz der Antwort", bewertet wie relevant die Antwort für die Eingabe ist. Hier werden Antworten mit überflüssigen Informationen oder unvollständige Antworten geringer bewertet und Antworten die die Frage vollständig beantworten und keine überflüssigen Informationen beinhalten, werden höher bewertet. Der Wert der Relevanz der Antwort kann zwischen -1 und 1, ist in der Praxis aber meistens zwischen 0 und 1 [Ragb].

Die Formel die hierfür verwendet wird ist:

$$\text{answer relevancy} = \frac{1}{N}\sum_{1}^{N}\cos(E_g i, E_0)$$

Answer semantic similarity

Bei der Metrik "Answer semantic similarity", zu Deutsch "Semantische Ähnlichkeit der Antwort", werden die `answer` und `ground_truth` miteinander verglichen. Hierbei wird die Semantische Ähnlichkeit der beiden Werte gemessen. Dieser Wert ist eine Gleitkommazahl zwischen 0 und 1. Ein höherer Wert steht hierbei für eine höhere Genauigkeit. Hierfür wird das embedding Model verwendet um die `answer` und `ground_truth` in Vektoren einzubetten und diese werden dann mittels Kosinus-Ähnlichkeit verglichen. Die Formel hierfür ist in 2.5 bereits vorgestellt worden [Ragc].

Answer correctness

Bei der Metrik "Answer correctness", zu Deutsch "Korrektheit der Antwort", wird die Korrektheit der Antwort überprüft. Dieser Wert ist ebenfalls eine Gleitkommazahl zwischen 0 und 1. Höhere Werte bedeuten hier eine höhere Korrektheit. Hierfür werden `ground_truth` und `answer` miteinander verglichen. Hierfür wird einmal die Semantische Ähnlichkeit

Model Bewertung				
Metrik	llama3	mistral	phi3	gemma2
answer_relevancy	0.9998	0.9802	0.9981	0.7481
answer_correctness	0.6126	0.3717	0.4172	0.5011
answer_similarity	0.9032	0.8935	0.8992	0.8712
faithfulness	0.7415	0.8444	0.6852	0.5938

Abbildung 6.1 Ragas Ergebnisse

betrachtet und dazu die faktische Ähnlichkeit. Diese Werte werden im Anschluss mittels einer gewichtet Formel benutzt um den Answer correctness Score zu erhalten.

$$\text{F1 Score} = \frac{|\text{TP}|}{(|\text{TP}| + 0.5 * (|\text{FP}| + |\text{FN}|))}$$

Wobei TP Richtig positive Fakten sind, Werte die laut Antwort wahr sind und auch in der `ground_truth` enthalten sind. FP sind falsch positive Fakten. Fakten die laut `answer` wahr sind aber nicht in der `ground_truth` enthalten sind. FN sind falsch negative Fakten. Fakten die in der `ground_truth` enthalten sind, aber nicht in der `answer` [Raga].

6.2.3 Ergebnisse

Nach Erstellen von mehreren Testsets mit verschiedene Modellen sind bei einer `test_size` von 10 jedes mal mehrere Reihen in der `ground_truth` Spalte leer geblieben. Deshalb wurde ein Testset verwendet, welches 8 Reihen hatte, bei denen alle Werte gefüllt waren.

In der Tabelle 6.1 sind die Ergebnisse des Ragas Benchmarks dargestellt. Die verschiedenen Metriken die betrachtet wurden sind `answer_relevancy`, `answer_correctness`, `answer_similarity` und `faithfulness`. Alle Metriken außer `faithfulness` sind am höchsten bei Llama3. `Faithfulness` hat den höchsten Wert bei Mistral.

Das beste Modell in diesem Test ist Llama3 mit 3 Metriken, bei denen es am besten bewertet wurde. Das zweitbeste ist Phi-3 mit zwei Metriken als zweitbestes Modell und zwei als drittes. Das drittbeste Modell ist Mistral mit einer Bewertung als bestes Modell, zwei als drittes und eins als viertes. Das schlechteste Modell innerhalb dieser Ragas Bewertung ist Gemma2 mit einer Metrik als zweitbestes und drei als viertes.

Der größte Unterschied zwischen Llama3 und den anderen Modellen ist die Menge an Trainingsdaten, die verwendet wurden. Mit 15 Billionen Token ist dies 7 Billionen von den für Gemma 2 verwendet 8 Billionen Token entfernt. Das zweitbeste Modell in diesem Vergleich, Phi-3, hatte die kleinste bekannte Menge an Trainingsdaten und war dafür mit 14 Milliarden Parametern das größte Modell. Dies zeigt, dass die Platzierung der Modelle hier weder direkt von der Menge an Trainingsdaten abhängt, noch von der Menge an Parametern.

6.3 Umfrage Vergleich

Dieser Schritt der Bewertung greift Anforderungen A4 und A7 erneut auf. A4 wurde bereits durch die RAG Pipeline und den Importer in den Chatbot eingebaut, der nächste

Schritt ist es zu überprüfen, ob die richtigen Unternehmensdaten verwendet werden und diese verständlich und logisch in die Antworten mit eingebaut werden. Hierfür wurde 4 Mitarbeitern aus dem Unternehmen ein Fragebogen zum Ausfüllen gegeben und dazu wurde ihnen Zugang zum System freigeschaltet. Dies bewertet gleichzeitig die Antworten und ermöglicht es die 2 Modelle miteinander zu vergleichen.

6.3.1 Fragebogen

Der Fragebogen wurde so erstellt, dass es keine genauen Fragen gibt, sondern nur Themengebiete, die abgefragt werden sollen. Das wichtige beim Auswählen der Themengebiete war, dass es Informationen waren, die auf der Confluence Instanz, auf die der Chatbot Zugriff hatte, verfügbar waren. Die Confluence Instanz die für diese Arbeit verwendet wurde ist ein Testsystem auf das für diese Arbeit verschiedene Blogeinträge und Dokumentationen geladen wurden, die den Einträgen auf dem Produktivsystem ähneln. Die Fragen für den Fragebogen werden aus diesen neu erstellten Einträgen erstellt.

Folgende Aufgaben wurden den Teilnehmern gegeben:

1. Wie richten sie die VPN ein? Lassen sie sich eine Anleitung dazu ausgeben.
2. Wie lösen Sie bei aktiven VPN zur TD das Problem von DNS Auflösung?
3. Welche Möglichkeiten gibt es zum Teilen von sensitiven Daten?

Die den Teilnehmern gegeben Aufgaben decken unterschiedliche Kompetenzen des Chatbots ab.

Frage 1, die Einrichtung der VPN: Der Chatbot sollte in der Lage sein, eine klare, schrittweise Anleitung zur VPN-Einrichtung aus dem Confluence-Wissen zu extrahieren und dem Teilnehmer zu präsentieren.

Frage 2, DNS-Auflösungsproblem bei aktivem VPN: Hier sollte der Chatbot Problemlösungsstrategien aus dem Confluence ableiten und dem Benutzer mögliche Lösungsansätze vorschlagen.

Frage 3, Teilen sensitiver Daten: Der Chatbot sollte verschiedene in Confluence dokumentierte Methoden zum sicheren Teilen von sensitiven Daten erkennen und dem Benutzer erklären können.

Die Teilnehmer sollten die Antworten auf einer Skala von 1-5 für beide LLMs bewerten, wobei 1 für sehr schlecht und 5 für sehr gut steht.

6.3.2 Fragebogen Ergebnisse

In A.3 sind die durchschnittlichen Bewertungen der LLMs dargestellt. Die Ergebnisse zeigen, dass Llama3 und Phi-3 bei zwei der drei Fragen identische Durchschnittswerte erreichten. Bei der zweiten Frage übertraf Llama3 Phi-3 um 0,25 Punkte.

Konkret erzielten beide Modelle bei der Erstellung der Anleitung und bei der Frage zum sicheren Datenaustausch Bewertungen, die etwas besser als `Okay` ausfielen. Bei der Frage zur Problemlösungsstrategie wurde Llama3 als `Okay` eingestuft, während Phi-3 leicht darunter lag.

Dies führt dazu, dass Llama3 insgesamt die bessere Bewertung hat und somit im Vergleich besser ist. Genau wie in 6.2.3 bereits erwähnt, ist der größte Unterschied zwischen Llama3

und den anderen Modellen die Menge an für das Trainierer verwendeten Daten. Da Phi-3 besser als Gemma 2 im Ragas Vergleich abgeschlossen hat lässt sich die Abhängigkeit der Platzierung von der Menge der zum Training verwendeten Token innerhalb dieses Kontexts widerlegen.

7 Fazit

Die in dieser Bachelorarbeit erzielten Ergebnisse bieten wichtige Erkenntnisse für die Entwicklung und den Einsatz fortschrittlicher Chatbot-Systeme. Diese Erkenntnisse haben nicht nur akademischen Wert, sondern können auch als Grundlage für zukünftige Entwicklungen und Implementierungen dienen.

7.1 Zusammenfassung Konzeption und Entwicklung

Die Konzeption des Chatbots hat gezeigt, dass es bereits heute verschiedene Möglichkeiten für die Entwicklung eines Chatbots gibt. Jeder Teil des Systems von der Datenbank, bis zum Framwork, hat verschiedene Alternativen die alle ihre eigenen Vorteile bieten.

In dieser Arbeit wurde ein Konzept für einen RAG-Chatbot entwickelt. Dieses wurde prototypisch mit dem Framework Langchain implementiert und in Google Chat eingebaut. Dieser Übergang von theoretischem Konzept zur prototypischen Implementierung hat das Ziel der Überbrückung von Theorie und Praxis der Arbeit erfüllt.

Die Entwicklung zeigte, dass moderne Frameworks und Tools die Erstellung KI-gestützter Chatbots erheblich erleichtern, wobei dennoch eine sorgfältige Auswahl der Komponenten erforderlich ist, um ein optimales Ergebnis zu erreichen. Die Entwicklung des Chatbots hat das Ziel der praxisorientierten Lösung für ein Chatbot System innerhalb dieser Arbeit erfüllt.

7.2 Zusammenfassung Bewertung

Im Rahmen der Bewertung wurden vier verschiedene LLMs miteinander verglichen, wobei ein zweistufiger Evaluierungsprozess angewandt wurde. Im ersten Schritt wurde ein Vergleich über das Framework RAGAS durchgeführt. RAGAS ermöglichte eine objektive Bewertung verschiedener Aspekte der Modellleistung.

Im zweiten Schritt wurde ein Vergleich mittels eines Fragebogens durchgeführt. Dieser Ansatz erlaubte es, die Nutzererfahrung und die praktische Anwendbarkeit der verschiedenen Modelle zu bewerten.

Die Ergebnisse zeigten, dass LLaMA 3 in beiden Bewertungsphasen am besten Abschnitt. Diese Bewertung ermöglichte die Identifikation des am besten geeigneten LLMs unter den betrachteten Optionen für diesen Anwendungsfall.

7.3 Ausblick

Die Erkentnisse dieser Arbeit eröffnen mehrere Richtungen für zukünftige Forschung und Entwicklung.

Die erfolgreiche Implementierung des RAG-Chatbots zeigt das Potenzial dieser Technologie. In der Bewertung mit Ragas hat sich gezeigt, dass es noch Potenzial für Verbesserung gibt.

Zukünftige Arbeiten könnten sich auf die Verbesserung des RAG-Prozesses konzentrieren, um noch präzisere und kontextrelevantere Informationen zu liefern.

Die genutzen Modelle innerhalb dieser Arbeit waren die Standard Varianten, die von diesen Modellen existieren. Zukünftig könnten sich Arbeiten darauf fokussieren, mit finetuned Modellen zu arbeiten, die auf eine bestimmte Domäne oder Sprache finetuned sind um präzisere und weniger allgemeine Modelle zu bewerten. Dazu könnten Modelle in anderen Größen betrachtet werden.

Der Vergleich mit zwei verschiedenen Methoden erwies sich als sinnvoll, damit die menschliche Komponente bei der Kommunikation auch beachtet wird. Zukünftige Arbeiten könnten diesen Ansatz standardisieren und erweitern um einen aussagekräftigeren Vergleich zwischen verschiedenen Chatbots zu ermöglichen und vereinfachen.

Eine weitere Richtung in die zukünftige Entwicklung und Forschung gehen könnte ist die Erweiterung der Funktionalität des RAG-Chatbots mit einem multimodalen Ansatz.

Innerhalb dieser Arbeit wurden die verschiedenen Modelle mit dem selben Prompt bedient. Der Einsatz von verschiedenen Prompts bei den verschiedenen Modellen könnte einen Einfluss auf ihre Ausgaben haben, was in einer weiteren Arbeit betrachtet werden könnte.

Insgesamt hat diese Bachelorarbeit nicht nur zur Entwicklung und Bewertung eines fortschrittlichen RAG-Chatbots beigetragen, sondern auch wichtige Grundlagen für zukünftige Forschungen und Anwendungen in diesem sich rasch entwickelnden Bereich der künstlichen Intelligenz gelegt.

A Anhang


```
{
"type": "MESSAGE",
"eventTime": "2024-07-04T18:57:11.735759Z",
"message": {
    "name": "spaces/****/messages/****",
    "sender": {
        "name": "users/****",
        "displayName": "Atakan Sentürk",
        "avatarUrl": "https://lh3.googleusercontent.com/a/**",
        "email": "a.sentuerk@****.de",
        "type": "HUMAN",
        "domainId": "****"
    },
    "createTime": "2024-07-04T18:57:11.735759Z",
    "text": "@Arthur Hi",
    "annotations": [
        {
            "type": "USER_MENTION",
            "startIndex": 0,
            "length": 7,
            "userMention": {
            "user": {
                "name": "users/****",
                "displayName": "Arthur",
                "avatarUrl": "https://lh5.googleusercontent.com/**",
                "type": "BOT"
            },
            "type": "MENTION"
            }
        }
    ],
    "thread": {
        "name": "spaces/****/threads/****",
        "retentionSettings": {
            "state": "PERMANENT"
    }
    },
    "space": {
        "name": "spaces/****",
        "type": "ROOM",
        "displayName": "Bot Test",
        "spaceThreadingState": "THREADED_MESSAGES",
        "spaceType": "SPACE",
        "spaceHistoryState": "HISTORY_ON",
        "spaceUri": "https://chat.google.com/room/**?cls=11"
```

```
    },
    "argumentText": " Hi",
    "threadReply": true,
    "retentionSettings": {
        "state": "PERMANENT"
    },
    "messageHistoryState": "HISTORY_ON",
    "formattedText": "@Arthur Hi"
},
"user": {
    "name": "users/****",
    "displayName": "Atakan Sentürk",
    "avatarUrl": "https://lh3.googleusercontent.com/a/**",
    "email": "a.sentuerk@****.de",
    "type": "HUMAN",
    "domainId": "****"
},
"space": {
    "name": "spaces/****",
    "type": "ROOM",
    "displayName": "Bot Test",
    "spaceThreadingState": "THREADED_MESSAGES",
    "spaceType": "SPACE",
    "spaceHistoryState": "HISTORY_ON",
    "spaceUri": "https://chat.google.com/room/***?cls=11"
},
"configCompleteRedirectUrl": "https://chat.google.com/api/**",
"common": {
    "userLocale": "de",
    "hostApp": "CHAT"
}
}
```


Abbildung A.0 JSON Nachricht der Google Chat API

```
{
    parent=req["message"]["space"]["name"],
    messageReplyOption='REPLY_MESSAGE_FALLBACK_TO_NEW_THREAD',

    body=
    {
        "text": "<"+user["name"]+"> "+ans["answer"],
        "thread": {"name": thread_name}
    }
}
```

Abbildung A.1 Asynchrones Message Objekt

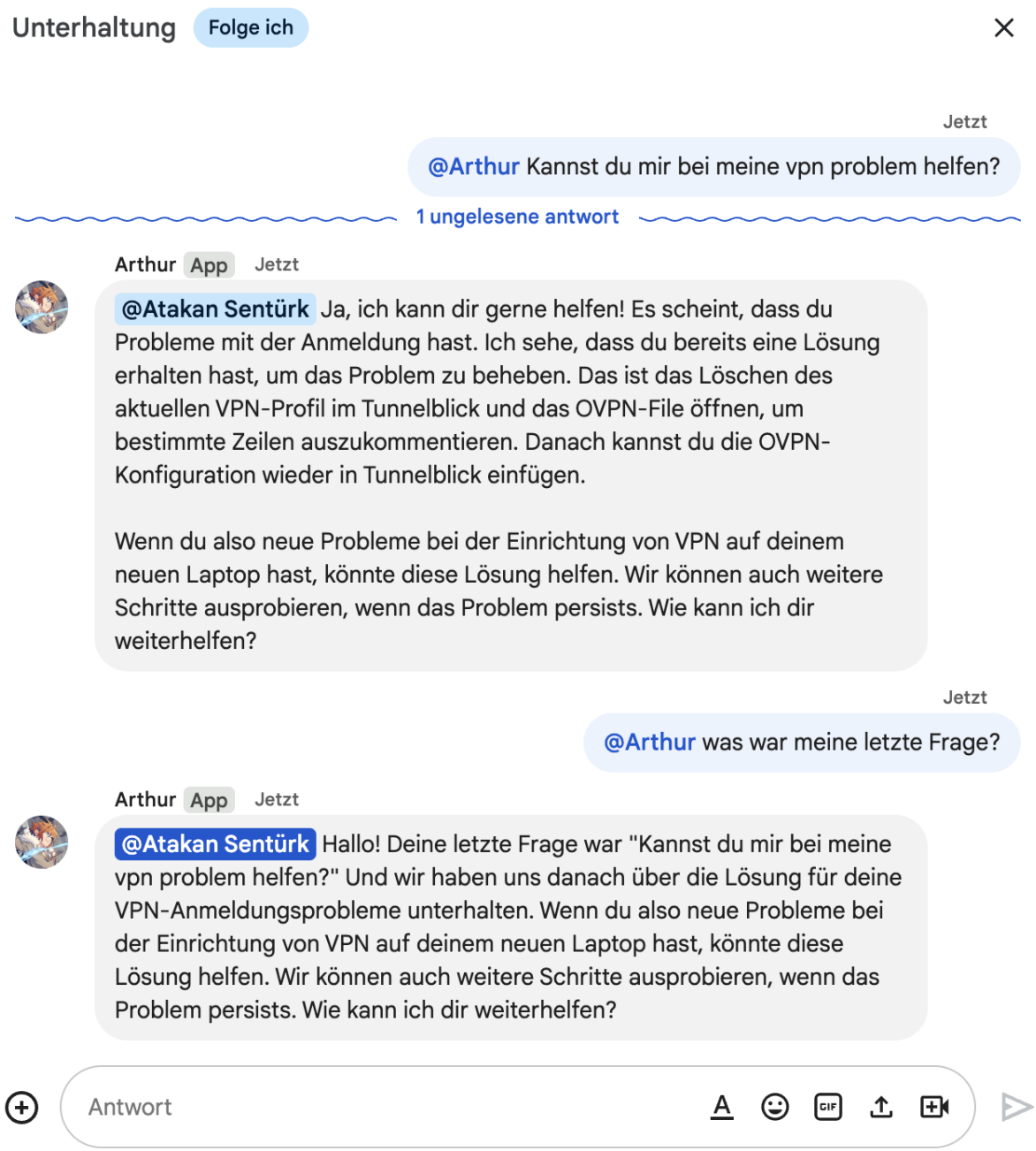


Abbildung A.2 Darstellung Threadinteraktion

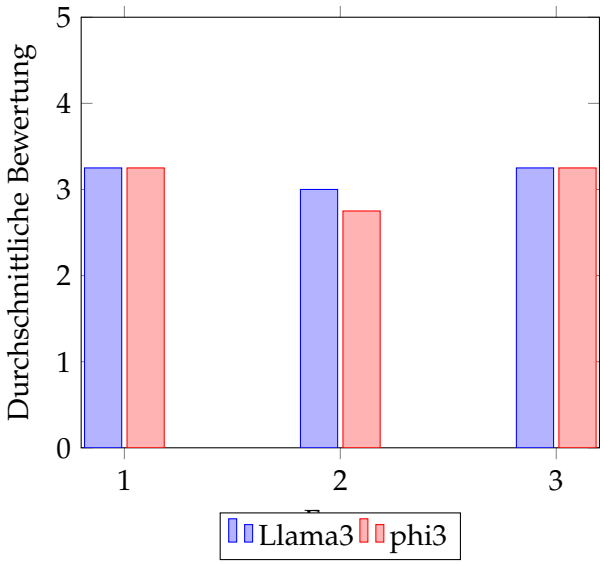


Abbildung A.3 Durchschnittliche Bewertungen von Llama3 und Phi3

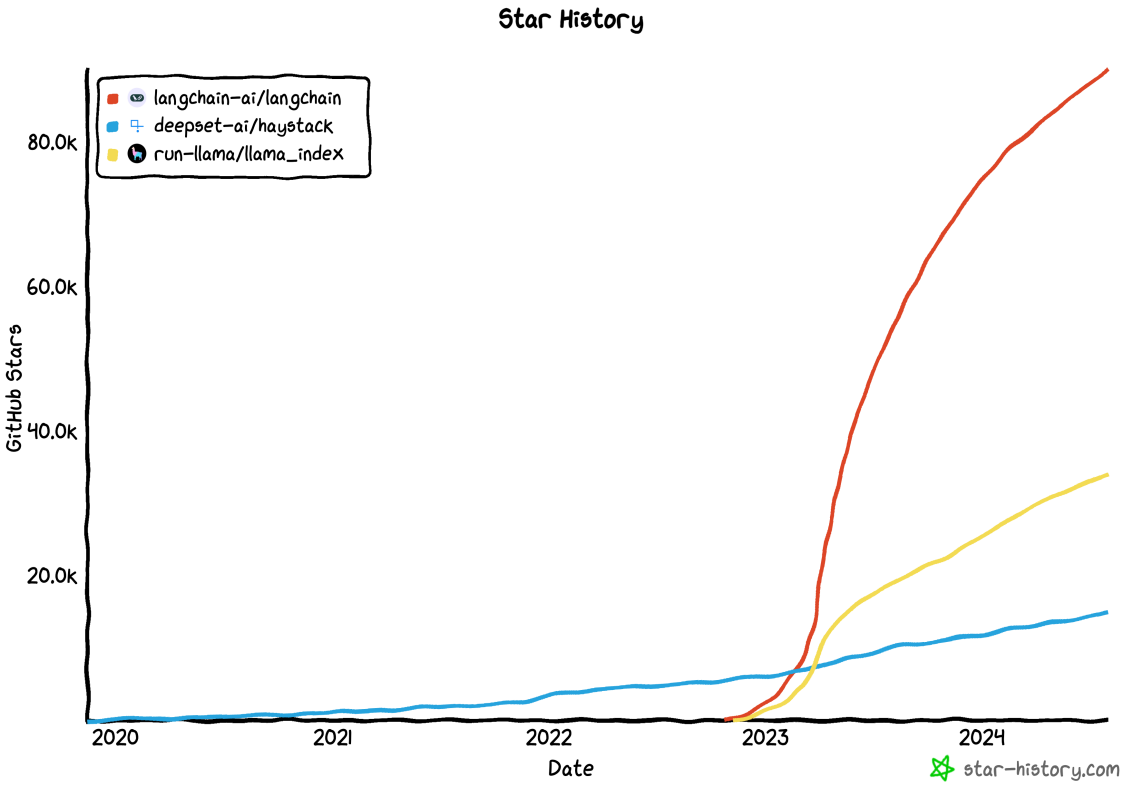


Abbildung A.4 Github Star Historie der Frameworks Langchain, Haystack und LlamaIndex

Literaturverzeichnis

[Abd24] M. Abdin, S. A. Jacobs, A. A. Awan, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, Q. Cai, M. Cai, C. C. T. Mendes, W. Chen, V. Chaudhary, D. Chen, D. Chen, Y.-C. Chen, Y.-L. Chen, P. Chopra, X. Dai, A. Del Giorno, G. de Rosa, M. Dixon, R. Eldan, V. Fragoso, D. Iter, M. Gao, M. Gao, J. Gao, A. Garg, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, J. Huynh, M. Javaheripi, X. Jin, P. Kauffmann, N. Karampatziakis, D. Kim, M. Khademi, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, C. Liu, M. Liu, W. Liu, E. Lin, Z. Lin, C. Luo, P. Madan, M. Mazzola, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacroce, S. Shah, N. Shang, H. Sharma, S. Shukla, X. Song, M. Tanaka, A. Tupini, X. Wang, L. Wang, C. Wang, Y. Wang, R. Ward, G. Wang, P. Witte, H. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, S. Yadav, F. Yang, J. Yang, Z. Yang, Y. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang und X. Zhou. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone, Mai 2024. (besucht am 2024-07-31), comment: 19 pages.

[Ada20] E. Adamopoulou und L. Moussiades. Chatbots: History, Technology, and Applications. *Machine Learning with Applications*, 2:100006, Dez. 2020. (besucht am 2024-05-10).

[Ain23] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón und S. Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, Dez. 2023. (besucht am 2024-08-01), comment: Accepted at EMNLP 2023. Added to related work.

[Ala] J. Alammar. The Illustrated Transformer. `https://jalammar.github.io/illustrated-transformer/`. (besucht am 2024-06-23).

[Ama] 2022 Echo Dot 5th Gen Smart Speaker | Charcoal | Amazon. `https://www.amazon.com/All-New-release-Smart-speaker-Charcoal/dp/B09B8V1LZ3`. (besucht am 2024-08-02).

[Ama24] X. Amatriain. Prompt Design and Engineering: Introduction and Advanced Methods, Mai 2024. (besucht am 2024-06-20).

[Bah16] D. Bahdanau, K. Cho und Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate, Mai 2016. (besucht am 2024-05-19), comment: Accepted at ICLR 2015 as oral presentation.

[Bel20] I. Beltagy, M. E. Peters und A. Cohan. Longformer: The Long-Document Transformer, Dez. 2020. (besucht am 2024-08-01), comment: Version 2 introduces the Longformer-Encoder-Decoder (LED) model.

[Ber24] L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak und O. Evans. The Reversal Curse: LLMs Trained on "A Is B"Fail to Learn "B Is A", Mai 2024. (besucht am 2024-06-26), comment: 21 pages, 11 figures.

[Cha] ChatGPT. `https://chatgpt.com/`. (besucht am 2024-08-02).

[Cha22a] D. Chandrasekaran und V. Mago. Evolution of Semantic Similarity – A Survey. *ACM Computing Surveys*, 54(2):1–37, März 2022. (besucht am 2024-06-13), comment: 29 pages, 5 figures, submitted to ÄCM Computing Survey".

[Cha22b] H. Chase. LangChain. `https://github.com/langchain-ai/langchain`, Okt. 2022. (besucht am 2024-06-20).

[Cha24] ChatGPT. `https://apps.apple.com/us/app/chatgpt/id6448311069`, Juli 2024. (besucht am 2024-08-02).

[Chi24] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez und I. Stoica. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference, 2024.

[Chr] Chroma. `https://www.trychroma.com/`. (besucht am 2024-07-28).

[Cla] Claude. `https://claude.ai/new`. (besucht am 2024-07-22).

[Cra24] E. Craig, M. Pilanci, T. L. Menestrel, B. Narasimhan, M. Rivas, R. Dehghannasiri, J. Salzman, J. Taylor und R. Tibshirani. Pretraining and the Lasso, Apr. 2024. (besucht am 2024-08-01).

[Das24] S. Das. Fine Tune Large Language Model (LLM) on a Custom Dataset with QLoRA, Jan. 2024. (besucht am 2024-08-01).

[Dis] DistroKid Help Center. `https://support.distrokid.com/hc/en-us`. (besucht am 2024-07-31).

[Dis24a] DiscoResearch/Llama3-German-8B · Hugging Face. `https://huggingface.co/DiscoResearch/Llama3-German-8B`, Mai 2024. (besucht am 2024-06-25).

[Dis24b] DiscoResearch (Disco Research). ~urlhttps://huggingface.co/DiscoResearch, Mai 2024. (besucht am 2024-06-11).

[Fok] M. Fokina. The Future of Chatbots: 80+ Statistics (2024). `https://www.tidio.com/blog/chatbot-statistics/`. (besucht am 2024-07-27).

[Gal] Galaxy AI | Samsung Business US | Undefined Undefined. `https://www.samsung.com/us/business/galaxy-ai/`. (besucht am 2024-06-24).

[Gao24] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang und H. Wang. Retrieval-Augmented Generation for Large Language Models: A Survey, März 2024. (besucht am 2024-04-21), comment: Ongoing Work.

[Gem24] Gemma Team und Google DeepMind. Gemma 2: Improving Open Language Models at a Practical Size, Juni 2024.

[Gen] Generation with LLMs. https://huggingface.co/docs/transformers/en/llm_tutorial. (besucht am 2024-07-29).

[Ger24a] G. Gerganov. Ggerganov/Ggml, Juni 2024. (besucht am 2024-06-18).

[Ger24b] G. Gerganov. Ggerganov/Llama.Cpp, Juni 2024. (besucht am 2024-06-18).

[Giv] Giving Claude a Role with a System Prompt - Anthropic. https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/system-prompts. (besucht am 2024-08-01).

[Goo] Google Assistant, Your Own Personal Google. https://assistant.google.com/. (besucht am 2024-06-24).

[Goo16] I. Goodfellow, Y. Bengio und A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[Goo24] Google Chat. *Wikipedia*, Juli 2024. (besucht am 2024-08-01).

[Gup] V. Gupta. Block Sparse Attention Huggingface. https://huggingface.co/blog/big-bird. (besucht am 2021-03-31).

[Han23] Y. Han, C. Liu und P. Wang. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge, Okt. 2023. (besucht am 2024-04-04).

[Hay] Deepset-Ai/Haystack: :Mag: LLM Orchestration Framework to Build Customizable, Production-Ready LLM Applications. Connect Components (Models, Vector DBs, File Converters) to Pipelines or Agents That Can Interact with Your Data. With Advanced Retrieval Methods, It's Best Suited for Building RAG, Question Answering, Semantic Search or Conversational Agent Chatbots. https://github.com/deepset-ai/haystack?tab=readme-ov-file. (besucht am 2024-06-19).

[Hea08] J. Heaton. *Introduction to Neural Networks with Java*. Heaton Research, St. Louis, Mo, 2. ed., 1. print Aufl., 2008.

[Het] Dedicated Server Hosting. https://www.hetzner.com/dedicated-rootserver/matrix-gpu/. (besucht am 2024-07-14).

[Hig] The High-Performance Vector Database Built for Scale | Milvus. https://milvus.io/. (besucht am 2024-07-28).

[Hua] A. Huang. The Loebner Prize. https://www.ocf.berkeley.edu/{~}arihuang/academic/research/loebner.html. (besucht am 2024-07-31).

[Hua23] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin und T. Liu. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions, Nov. 2023. (besucht am 2024-06-20), comment: Work in progress; 49 pages.

[Hug24] Hugging Face – The AI Community Building the Future. https://huggingface.co/, Juli 2024. (besucht am 2024-08-01).

[Ind] Indexing & Embedding - LlamaIndex. https://docs.llamaindex.ai/en/stable/understanding/indexing/indexing/. (besucht am 2024-08-01).

[Inta] Intfloat/Multilingual-E5-Large-Instruct · Hugging Face. https://huggingface.co/intfloat/multilingual-e5-large-instruct. (besucht am 2024-08-01).

[Intb] Introducing LlamaCloud and LlamaParse — LlamaIndex, Data Framework for LLM Applications. https://www.llamaindex.ai/blog/introducing-llamacloud-and-llamaparse-af8cedf9006b. (besucht am 2024-06-20).

[Intc] Introducing Meta Llama 3: The Most Capable Openly Available LLM to Date. https://ai.meta.com/blog/meta-llama-3/. (besucht am 2024-06-23).

[Intd] Introduction | LangChain. https://python.langchain.com/v0.2/docs/introduction/. (besucht am 2024-06-20).

[Kar] Karpathy (Andrej). https://github.com/karpathy. (besucht am 2024-06-26).

[Kem23] C. Kempkes. Das Neuron Und Die Aktivierungsfunktion — Grundlagen Und Beispiele an Logischen Bausteinen, Apr. 2023. (besucht am 2024-06-01).

[Koh] V. Kohli. What Is a Context Window? https://www.techtarget.com/whatis/definition/context-window. (besucht am 2024-07-29).

[Kri22] S. Krishnan. How Do Determine the Number of Layers and Neurons in the Hidden Layer?, Aug. 2022. (besucht am 2024-06-01).

[LaC] K. LaCurts. Criticisms of the Turing Test and Why You Should Ignore (Most of) Them.

[Lana] LangChain. https://www.langchain.com/. (besucht am 2024-08-01).

[Lanb] LangChain. https://www.langchain.com/langchain. (besucht am 2024-08-01).

[Lanc] LangServe | LangChain. https://python.langchain.com/v0.2/docs/langserve/. (besucht am 2024-08-01).

[Land] LangSmith. https://www.langchain.com/langsmith. (besucht am 2024-08-01).

[Lan23] C. Lanquillon und S. Schacht. *Knowledge Science – Grundlagen: Methoden der Künstlichen Intelligenz für die Wissensextraktion aus Texten*. Springer Fachmedien Wiesbaden, Wiesbaden, 2023. (besucht am 2024-06-03).

[Lee24] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoeybi, B. Catanzaro und W. Ping. NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models, Mai 2024. (besucht am 2024-06-20).

[Leo] LeoLM/HellaSwag_de · Datasets at Hugging Face. https://huggingface.co/datasets/LeoLM/HellaSwag_de. (besucht am 2024-06-26).

[Lew21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel und D. Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Apr. 2021. (besucht am 2024-06-09), comment: Accepted at NeurIPS 2020.

[Liu22] J. Liu. LlamaIndex, Nov. 2022. (besucht am 2024-06-20).

[Llaa] Llama2.c/Run.c at Master · Karpathy/Llama2.c. https://github.com/karpathy/llama2.c/blob/master/run.c. (besucht am 2024-06-26).

[Llab] Llama3/MODEL_CARD.Md at Main · Meta-Llama/Llama3. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md. (besucht am 2024-06-23).

[Llac] LlamaCloud - LlamaIndex. https://docs.llamaindex.ai/en/stable/llama_cloud/. (besucht am 2024-06-20).

[Llad] LlamaIndex - LlamaIndex. https://docs.llamaindex.ai/en/stable/. (besucht am 2024-06-20).

[LMS] LM Studio - Discover and Run Local LLMs. https://lmstudio.ai. (besucht am 2024-06-26).

[Met24] Building Meta's GenAI Infrastructure, März 2024. (besucht am 2024-06-23).

[Mik13] T. Mikolov, I. Sutskever, K. Chen, G. Corrado und J. Dean. Distributed Representations of Words and Phrases and Their Compositionality, Okt. 2013. (besucht am 2024-06-20).

[Mis] Mistral. https://huggingface.co/docs/transformers/en/model_doc/mistral. (besucht am 2024-07-31).

[Mor13] B. Morais. Can Humans Fall in Love with Bots? *The New Yorker*, Nov. 2013. (besucht am 2024-05-16).

[MTE] MTEB Leaderboard - a Hugging Face Space by Mteb. https://huggingface.co/spaces/mteb/leaderboard. (besucht am 2024-06-20).

[Mue23] N. Muennighoff, N. Tazi, L. Magne und N. Reimers. MTEB: Massive Text Embedding Benchmark, März 2023. (besucht am 2024-06-20), comment: 24 pages, 14 tables, 6 figures.

[Nav24] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes und A. Mian. A Comprehensive Overview of Large Language Models, Apr. 2024. (besucht am 2024-06-20).

[NVI] NVIDIA Glossary: What Is a Vector Database? https://www.nvidia.com/en-eu/glossary/vector-database/. (besucht am 2024-06-20).

[Oll24] Ollama/Ollama. Ollama, Juni 2024. (besucht am 2024-06-20).

[O'S15] K. O'Shea und R. Nash. An Introduction to Convolutional Neural Networks, Dez. 2015. (besucht am 2024-07-29), comment: 10 pages, 5 figures.

[Pen24] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. V. Werra und T. Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, 2024.

[Per21] T. Perry. What Is Tokenization in Natural Language Processing (NLP)?, Febr. 2021. (besucht am 2024-06-12).

[Raga] Answer Correctness | Ragas. https://docs.ragas.io/en/stable/concepts/metrics/answer_correctness.html. (besucht am 2024-08-02).

[Ragb] Answer Relevance | Ragas. https://docs.ragas.io/en/stable/concepts/metrics/answer_relevance.html. (besucht am 2024-08-02).

[Ragc] Answer Semantic Similarity | Ragas. https://docs.ragas.io/en/stable/concepts/metrics/semantic_similarity.html. (besucht am 2024-08-02).

[Ragd] Faithfulness | Ragas. https://docs.ragas.io/en/stable/concepts/metrics/faithfulness.html. (besucht am 2024-08-02).

[Rai24] L. Rainie und J. Anderson. Experts Imagine the Impact of Artificial Intelligence by 2040, Febr. 2024.

[Saf] Safetensors. https://huggingface.co/docs/safetensors/en/index. (besucht am 2024-06-19), accessed on 2024-06-19 17:25:58.

[Ser] Serverless Inference API. https://huggingface.co/docs/api-inference/index. (besucht am 2024-06-09).

[Sha19] N. Shazeer. Fast Transformer Decoding: One Write-Head Is All You Need, Nov. 2019. (besucht am 2024-08-01).

[Sir] Siri. https://www.apple.com/siri/. (besucht am 2024-06-24).

[Sta] Stack Overflow and OpenAI Partner to Strengthen the World's Most Popular Large Language Models - Press Release - Stack Overflow. https://stackoverflow.co/company/press/archive/openai-partnership/. (besucht am 2024-06-25).

[Sut14] I. Sutskever, O. Vinyals und Q. V. Le. Sequence to Sequence Learning with Neural Networks, Dez. 2014. (besucht am 2024-06-05), comment: 9 pages.

[Tou] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave und G. Lample. LLaMA: Open and Efficient Foundation Language Models.

[Uns] Unser Puls schlägt digital. https://www.techdivision.com/. (besucht am 2024-07-31).

[Vas23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser und I. Polosukhin. Attention Is All You Need, Aug. 2023. (besucht am 2024-04-21), comment: 15 pages, 5 figures.

[Vec] Vector Stores | LangChain. https://python.langchain.com/v0.2/docs/integrations/vectorstores/. (besucht am 2024-07-28).

[Ven21] C. B. Vennerød, A. Kjærran und E. S. Bugge. Long Short-term Memory RNN, Mai 2021. (besucht am 2024-07-29).

[Wal09] R. S. Wallace. The Anatomy of a.L.I.C.E. In R. Epstein, G. Roberts und G. Beber, Hg., *Parsing the Turing Test: Philosophical and Methodological Issues in the Quest for the Thinking Computer*, S. 181–210. Springer Netherlands, Dordrecht, 2009.

[Wan24a] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder und F. Wei. Multilingual E5 Text Embeddings: A Technical Report, Febr. 2024. (besucht am 2024-07-28), comment: 6 pages.

[Wan24b] X. Wang, M. Salmani, P. Omidi, X. Ren, M. Rezagholizadeh und A. Eshaghi. Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models, Mai 2024. (besucht am 2024-07-29), comment: Accepted to IJCAI 2024 Survey Track – camera-ready version.

[Wei66] J. Weizenbaum. ELIZA—a Computer Program for the Study of Natural Language Communication between Man and Machine. Jan. 1966.

[Whaa] What Is a Large Language Model (LLM)? `https://www.cloudflare.com/learning/ai/what-is-large-language-model/`. (besucht am 2024-08-01).

[Whab] What Is Haystack? `https://haystack.deepset.ai/overview/intro/`. (besucht am 2024-06-19).

[Whac] What Is RNN? - Recurrent Neural Networks Explained - AWS. `https://aws.amazon.com/what-is/recurrent-neural-network/`. (besucht am 2024-07-29).

[Whad] What Is Semantic Search? `https://docs.cohere.com/docs/what-is-semantic-search`. (besucht am 2024-06-20).

[Wha21] What Is NLP (Natural Language Processing)? | IBM. `https://www.ibm.com/topics/natural-language-processing`, Sept. 2021. (besucht am 2024-06-16).